D^2Prune : Sparsifying Large Language Models via Dual Taylor Expansion and Attention Distribution Awareness

Lang Xiong[1*], Ning Liu[2*], Ao Ren[1†], Yuheng Bai[1], Haining Fang[1], Binyan Zhang[1], Zhe Jiang[1], Yujuan Tan[2,1†], Duo Liu[1]

[1]Chongqing University, China
[2]National University of Defense Technology, China
[3]Beijing Innovation Center of Humanoid Robotics, China

langxiong@stu.cqu.edu.cn, ningliu1220@gmail.com, {ren.ao, yuheng, haining.fang, zby, jiangzhe}@cqu.edu.cn, tanyujuan@nudt.edu.cn, liuduo@cqu.edu.cn

Abstract

Large language models (LLMs) face significant deployment challenges due to their massive computational demands. While pruning offers a promising compression solution, existing methods suffer from two critical limitations: (1) They neglect activation distribution shifts between calibration data and test data, resulting in inaccurate error estimations; (2) Overlooking the long-tail distribution characteristics of activations in the attention module. To address these limitations, this paper proposes a novel pruning method, D^2Prune. First, we propose a dual Taylor expansion-based method that jointly models weight and activation perturbations for precise error estimation, leading to precise pruning mask selection and weight updating and facilitating error minimization during pruning. Second, we propose an attention-aware dynamic update strategy that preserves the long-tail attention pattern by jointly minimizing the KL divergence of attention distributions and the reconstruction error. Extensive experiments show that D^2Prune consistently outperforms SOTA methods across various LLMs (e.g., OPT-125M, LLaMA2/3, Qwen3). Moreover, the dynamic attention update mechanism also generalizes well to ViT-based vision models like DeiT, achieving superior accuracy on ImageNet-1K.

Introduction

In recent years, Large Language Models (LLMs) (Guo et al. 2025; Dubey et al. 2024; OpenAI 2023) have revolutionized the field of Natural Language Processing (NLP), excelling in complex tasks such as causal reasoning and text generation (Chi et al. 2024; Wang 2024; Mo et al. 2024; Li et al. 2024a). However, their exceptional performance comes at the cost of substantial memory and computational demands, posing significant challenges for deployment on resource-constrained devices (Li et al. 2024b; Kim et al. 2024). Extensive efforts have been made in model pruning techniques, aiming at shrinking the network size by removing redundant weights (Bai et al. 2024; Dong et al. 2024; Sun et al. 2023; Frantar and Alistarh 2023). By introducing sparsity, pruning enhances both memory and computational efficiency and has proven effective in LLMs. Nonetheless, traditional training-based model pruning generally requires substantial computational resources due to processes such as retraining (Liu et al. 2018; Blalock et al. 2020), training from scratch (Hoang and Liu 2023; Sreenivasan et al. 2022), or extensive iterative pruning (Chijiwa et al. 2021; Tanaka et al. 2020). These training-based methods are costly, especially for current large language models (LLMs). Recently, post-training pruning methods (Kwon et al. 2022; Zhang et al. 2024) have emerged as a more efficient schema for compressing LLMs as they require no additional training resources. These methods typically design weight importance metrics to remove redundant weights with minimal impact on layer-wise output errors, categorized into non-weight-update pruning (e.g., Magnitude (Han et al. 2015), Wanda (Sun et al. 2023), Pruner-Zero (Dong et al. 2024)) and weight-update pruning (e.g., LLM Surgeon (van der Ouderaa et al. 2023), SparseGPT (Frantar and Alistarh 2023), SparseLLM (Bai et al. 2024), ADMM-Grad (Boža 2024)). These methods fail to deliver desirable performance, especially at high sparsity regimes (70%-80%), as shown in Figure 1. Through in-depth analysis, we identify two fundamental limitations in current methodologies stemming from inadequate consideration of activation impacts on error propagation.

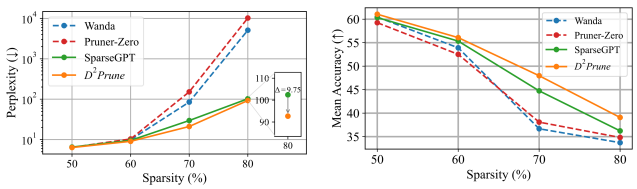
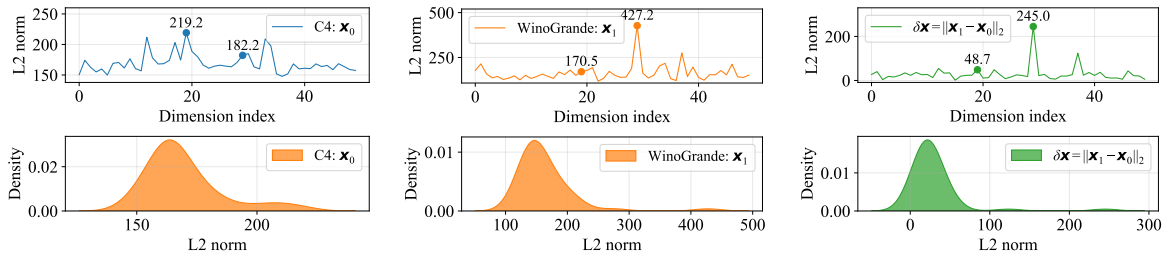


Figure 1: Pruning comparison for LLaMA-2-7B. Left: Perplexity for WikiText2 language modeling; Right: Mean accuracy for downstream tasks in Zero-shot.

First, the constant activation assumption in error estimation. Most mainstream pruning methods inherit the "constant activation assumption" from early methods like OBS (Hassibi, Stork, and Wolff 1993), which treat input activations as fixed during pruning. This assumption is valid for small-scale networks, where the train/test data is independent

*These authors contributed equally.
†Corresponding authors

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.



(a) Upstream activation distribution (b) Downstream activation distribution (c) Activation shift distribution

Figure 2: Activation distributions and their shift between upstream and downstream data in LLaMA-2-7B (layer 10). During pruning, a small subset of C4 (Raffel et al. 2020) is used as calibration data (upstream), while downstream tasks such as WinoGrande (Sakaguchi et al. 2021) are evaluated in a zero-shot setting. All inputs are formatted as 128-sample sequences with maximum embedding length. The L_2 norms represent the average activation magnitude across samples.

and identically distributed. It fails to hold in modern large language models (LLMs). Our experiments reveal significant shifts in input activation across datasets and layers. For example, Figure 2 illustrates a segment of activation features from the 10th layer of LLaMA-2-7B. The activation norms exhibit significant shifts, indicating a large deviation between the activations across datasets and layers.

Non-weight-update methods, such as Wanda and Pruner-Zero, evaluate importance via the product of weight (or gradient) magnitudes and activation norms, which essentially reflects output amplitude rather than true pruning error. Weight-update methods, such as SparseGPT and OBS, explicitly approximate the pruning-induced error via the single-variable-based Taylor expansion on the weight but omit the effect of activation variation between calibration and downstream data, causing inaccurate error modeling. Thus, the fixed activation assumption is fundamentally flawed for LLM pruning. A more accurate modeling of activation shift is required for precise error estimation and robust downstream performance.

Second, negligence of long-tail distribution in attention modules. Multi-head attention, as the core component of Transformers, shows long-tail attention distribution (as noted in several literatures (Zhou et al. 2021; Ji et al. 2021; Chen et al. 2022)) where a few key tokens dominate attention scores, which is pivotal for LLM reasoning. Nonetheless, current pruning methods treat query (Q), key (K), and value (V) weights in attention modules identically to linear layers (Gate, Up, and Down in MLP modules), incurring significant error propagation. Specifically, **the non-weight-update pruning accumulates errors in Q/K/V layers by retaining raw weights, while weight-update pruning disrupts the long-tail attention distribution through global weight adjustments.** As illustrated in Figure 3, direct updates to Q/K/V weights homogenize attention scores from the original long-tail distribution, severely distorting the model's focus on critical tokens. Conversely, skipping updates leads to significant errors in attention outputs due to error accumulation. This "all-or-nothing" approach fails to balance error compensation and distribution preservation, causing catastrophic performance drops, especially at high sparsity.

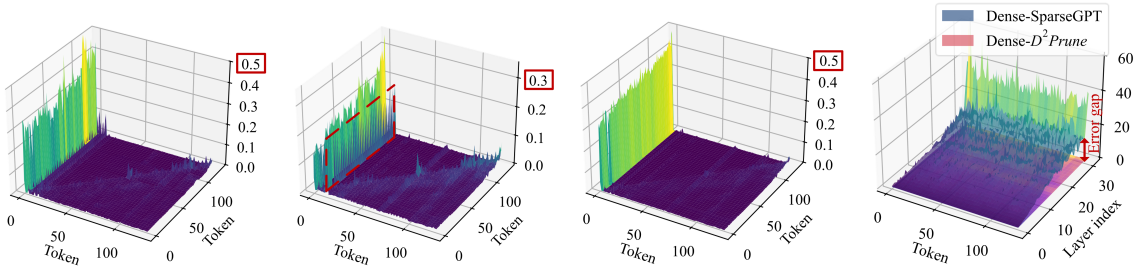
To tackle the above limitations, we propose D^2Prune, a novel pruning method for large language models via Dual Taylor Expansion and Attention Distribution Awareness. Specifically, to address the first limitation, we propose **Activation-Weight Dual-Sensitive Pruning Mechanism**. By formulating a Dual Taylor Expansion of the error function with respect to both activations and weights, our method jointly models the impacts of activation variations and weight perturbations on output errors, enabling precise pruning mask selection and weight updates. This dual expansion reduces perplexity by approximately 10% compared to conventional single-variable approaches , and it enhances accuracy at high sparsity by up to 40% on downstream tasks with significant distribution shifts . To address the second limitation, we propose **Attention Distribution-Aware Dynamic Weight Update Strategy**. Specifically, we formulate Q/K/V update states as a combinatorial optimization problem and propose a perplexity-guided lightweight adaptive search method that dynamically identifies update configurations. This strategy can effectively preserve the attention distribution while reducing the pruning errors by allowing weight updating. Experimental results demonstrate that this strategy reduces the root-mean-square error of the attention layer by an average of 61.2% compared to uniformly updating all q/k/v weights (Figure 3d).

Our contributions are summarized as follows:

- We propose a novel pruning framework (D^2Prune) for compressing LLMs, which simultaneously accounts for activation dynamics and weight sensitivity via Dual Taylor Expansion, enabling effective pruning mask selection and weight updating.
- We propose an attention distribution-aware dynamic weight update method, which dynamically identifies update configurations for q, k, v weights in attention modules, achieving balance in attention distribution preservation and pruning errors compensation.
- Experimental results demonstrate that D^2Prune achieves SOTA performance across varying parameter scales of LLMs (including OPT-125M, LLaMA2/3 models, Qwen3-8/14B.), with substantial improvements in perplexity and zero-shot accuracy across various sparsity levels, especially at high sparsity.

Preliminaries

Related Work. Post-training pruning has emerged as a critical technique for compressing Large Language Models



(a) Dense (b) SparseGPT (c) D^2Prune (d) Attention Error

Figure 3: Visualization of uniformized multi-head attention, and the root mean squared error of pruning across layers and input activation sequence in LLaMA-2-7B (with 128-token sequences from C4 as input). Figure 3a-3c show the attention distribution of the final layer for each method, while Figure 3d plots the RMSE of attention outputs for each layer after pruning.

(LLMs) (Kwon et al. 2022; Zhang et al. 2024; Reyhan, Rahnamayan, and Bidgoli 2024). In Appendix G, we review post-training pruning techniques in detail, categorizing them into the non-weight-update and weight-update pruning methods.

Problem Statement. Post-training pruning is performed by decomposing the full-model compression problem into layer-wise pruning subproblems, which can be written as:

$$\arg\min_{\boldsymbol{M}_l} ||\boldsymbol{W}_l\boldsymbol{X}_l - (\boldsymbol{M}_l \odot \boldsymbol{W}_l)\,\boldsymbol{X}_l||_2^2. \quad (1)$$

where the subscript l represents the l_{th} layer, X_l is the activation, W_l is the weight matrix, and $\boldsymbol{M}_l$ is the pruning mask corresponding to the weights, respectively. Furthermore, weight-update methods aim to adjust the remaining unpruned weights to compensate for the pruning-induced error, resulting in an updated weight matrix $\hat{\boldsymbol{W}}_l$. The optimization objective is as follows:

$$\arg\min_{\boldsymbol{M}_l,\hat{\boldsymbol{W}}_l} ||\boldsymbol{W}_l\boldsymbol{X}_l - \left(\boldsymbol{M}_l \odot \hat{\boldsymbol{W}}_l\right)\boldsymbol{X}_l||_2^2. \quad (2)$$

Nonetheless, simultaneously solving for both the sparsity mask and the adjustments to the remaining weights is an NP-hard problem. Mainstream approaches (Frantar and Alistarh 2023; Bai et al. 2024; Hassibi, Stork, and Wolff 1993) typically use a pruning metric to select the pruning mask and then optimize the remaining weights based on that mask. Prior studies mainly adopt the layer-wise pruning error minimization. They usually solve Eq. 1 and Eq. 2 through the single-variable Taylor expansion with respect to weights. OBS and SparseGPT calculate the inverse of the second-order derivative matrix (Hessian) to perform weight mask selection and update. Specifically, the output error change of a layer can be written in the single-variable Taylor expansion form in OBS and SparseGPT:

$$\delta E = \left(\frac{\partial E}{\partial \mathbf{w}}\right)^T \delta\mathbf{w} + \frac{1}{2}\delta\mathbf{w}^T\mathbf{H}\delta\mathbf{w}, \quad (3)$$

where $\mathbf{H} = \partial^2 E/\partial\mathbf{w}^2$, $\delta\mathbf{w} = \mathbf{w} - \mathbf{w}_0$, $\delta E = E(\mathbf{w}) - E(\mathbf{w}_0)$ represents the change of the objective function (error or loss change). Following the principle proposed in Optimal Brain Damage (LeCun, Denker, and Solla 1989), we define the saliency of a parameter as the increase in the objective function (i.e., loss) after removing that parameter. Therefore, the goal of pruning is to minimize the change in error given by Eq. 3, when setting one of the weights $\mathbf{w}$ to zero (represented as w_q). Elimating w_q is expressed as $\delta w_q + w_q = 0$ or expressed in vector form more generally, that is $\mathbf{e}_q^T\mathbf{w} + w_q = 0$, where $\mathbf{e}_q$ is the unit vector in weight space corresponding to scalar w_q. Then, the goal is to solve:

$$\delta\mathbf{w}^* = \arg\min_{\delta\mathbf{w}} \tfrac{1}{2}\,\delta\mathbf{w}^T\mathbf{H}\,\delta\mathbf{w} \quad \text{s.t.} \quad \mathbf{e}_q^T\,\delta\mathbf{w} + w_q = 0. \quad (4)$$

To solve the Eq. 4, we can form a Lagrangian function:

$$L_q = \frac{1}{2}\delta\mathbf{w}^T \cdot \mathbf{H} \cdot \delta\mathbf{w} + \lambda\left(\mathbf{e}_q^T\delta\mathbf{w} + w_q\right), \quad (5)$$

where λ is a Lagrange undetermined multiplier. Finally, we can obtain the optimal weight change $\delta\mathbf{w}$ and the change in error δE:

$$\delta\mathbf{w} = -\frac{w_q}{\left(\mathbf{H}^{-1}\right)_{qq}}\left(\mathbf{H}^{-1}\right)_{:,q}, \delta E = L_q = \frac{w_q^2}{\left(\mathbf{H}^{-1}\right)_{qq}}. \quad (6)$$

Since L_q is the increase in error and is used to reflect the "saliency" of w_q in prior studies, both the weight-update and non-weight-update pruning methods evaluate the importance of w_q utilizing the magnitude L_q or its variants. They rank weights by L_q and then prune the lowest-saliency weights to meet the target sparsity.

Methodology

We propose D^2Prune, a novel pruning framework for LLMs. An overview of D^2Prune is shown in Figure 4 and Algorithm 1 (available in Appendix A)

Dual Taylor Expansion for Mask Selection & Weight Update

Consider a pre-trained large language model where the original input activation of a certain layer is $\boldsymbol{x}_0$, the weight is $\boldsymbol{w}_0$, and the output is $y = {\boldsymbol{x}_0}^T\boldsymbol{w}_0$. Now, suppose we remove a certain weight such that the error is $E = ||\boldsymbol{x}^T\boldsymbol{w} - {\boldsymbol{x}_0}^T\boldsymbol{w}_0||_2^2$, where $\boldsymbol{w} = \boldsymbol{w}_0 + \delta\boldsymbol{w}$ denotes the weights retained after pruning, and $\boldsymbol{x}$ is the input activation. **Previous works typically sample a small calibration set and compute layerwise input activations during pruning, assuming the activations remain constant, i.e., $\boldsymbol{x} = \boldsymbol{x}_0$.** However, as

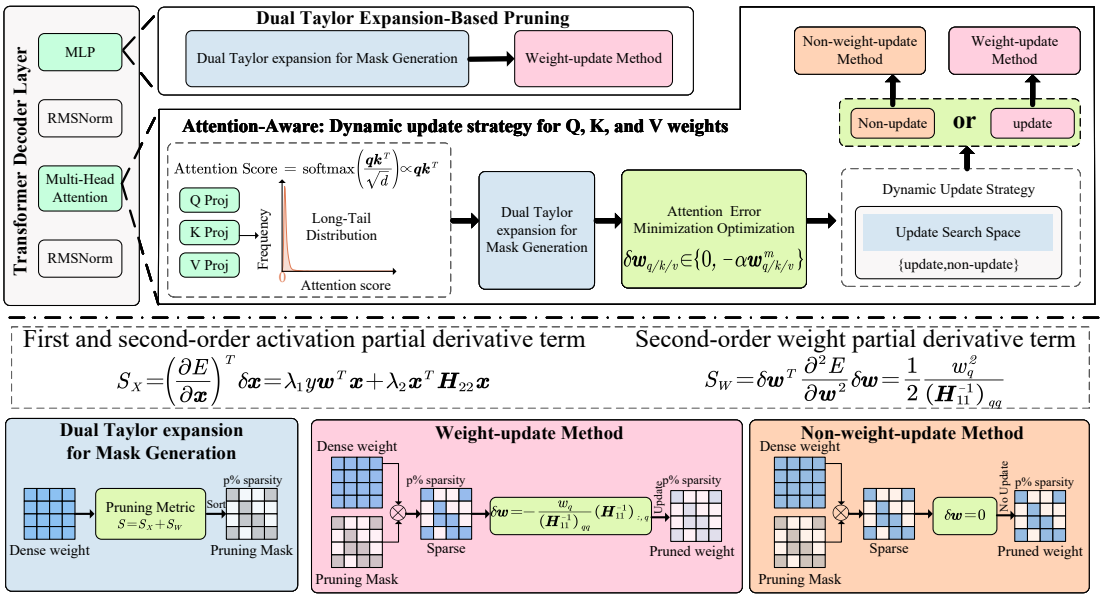


Figure 4: Illustration of the D^2Prune framework. Using the update status of q, k, and v weights in each attention layer as the search space, we take the minimization of ppl as the objective to dynamically adapt the update strategy of the q, k, and v weights.

demonstrated in Figure 2c, there exists a significant distributional shift between upstream and downstream activations. A more realistic formulation is $\boldsymbol{x} = \boldsymbol{x}_0 + \delta\boldsymbol{x}$, where $\delta\boldsymbol{x}$ is activation shift. Neglecting activation shift leads to inaccurate error estimation in prior pruning methods. To address this issue, we now consider the error minimization problem as a dual function of both weights and activations. The first-order partial derivative term with respect to the weights is 0 as a network trained to a local minimum in error, and the third and all higher-order terms converge to 0 and can be eliminated. Therefore, following OBS and SparseGPT (refer to Eq. 3), the functional Taylor series of the error change with respect to weights and activations is:

$$\delta E = \left(\frac{\partial E}{\partial \boldsymbol{u}}\right)^T \delta\boldsymbol{u} + \left(\frac{1}{2}\delta\boldsymbol{u}^T \boldsymbol{H}\delta\boldsymbol{u}\right), \tag{7}$$

where $\boldsymbol{u} = (\boldsymbol{w}, \boldsymbol{x})^T, \frac{\partial E}{\partial \boldsymbol{u}} = \left(\frac{\partial E}{\partial \boldsymbol{w}}, \frac{\partial E}{\partial \boldsymbol{x}}\right)^T, \delta\boldsymbol{u} = (\delta\boldsymbol{w}, \delta\boldsymbol{x})^T$, $\boldsymbol{H} = \begin{pmatrix} \partial^2 E/\partial \boldsymbol{w}^2 & \partial^2 E/\partial \boldsymbol{w}\,\partial \boldsymbol{x} \\ \partial^2 E/\partial \boldsymbol{x}\,\partial \boldsymbol{w} & \partial^2 E/\partial \boldsymbol{x}^2 \end{pmatrix}$. It is worth noting that, due to the presence of Layer Normalization in Transformers, the input activations of each layer are normalized during the forward propagation on the calibration data. Moreover, since the pruning-induced $\delta\boldsymbol{w}$ is small, the cross terms in the Hessian matrix become higher-order infinitesimals and have a negligible impact on error estimation (a theory derivation and experimental verification detailed in Appendix C). Therefore, it is reasonable to assume that the Hessian is predominantly dominated by its diagonal elements, such that:

$$\delta E = \left(\frac{\partial E}{\partial \boldsymbol{x}}\right)^T \delta\boldsymbol{x} + \frac{1}{2}\delta\boldsymbol{w}^T \boldsymbol{H}_{11}\delta\boldsymbol{w} + \frac{1}{2}\delta\boldsymbol{x}^T \boldsymbol{H}_{22}\delta\boldsymbol{x}, \tag{8}$$

where $\delta E = E(\boldsymbol{u}) - E(\boldsymbol{u}_0)$ represents the change of objective function (loss or error). Following the principle proposed in Optimal Brain Damage (Hassibi, Stork, and Wolff 1993), it is reasonable to define the saliency of a parameter as the increase in the objective function (i.e., loss) after removing that parameter, since objective functions play a central role in neural network pruning. Therefore, δE captures the change in the objective function (i.e., network error) caused by perturbations to the weights and activations, $\frac{\partial E}{\partial \boldsymbol{x}} = -\boldsymbol{w}\left(\boldsymbol{x}^T\boldsymbol{w} - y\right)$, $\boldsymbol{H}_{11} = \frac{\partial^2 E}{\partial \boldsymbol{w}^2} = \boldsymbol{x}\boldsymbol{x}^{\boldsymbol{T}}$, $\boldsymbol{H}_{\boldsymbol{22}} = \frac{\partial^2 E}{\partial \boldsymbol{x}^2} = \boldsymbol{w}\boldsymbol{w}^{\boldsymbol{T}}$. Since the activations of the batch training data are not available for each layer, computing $\delta\boldsymbol{x}$ directly is difficult. Fortunately, as shown in Figure 7-9 (detailed in Appendix C), we observe a linear correlation between activation norm shifts across different tasks and the calibration activations across model layers (while the relationship may appear nonlinear at the level of individual weight dimensions (see Figure 2), the layerwise nature of our error estimation makes the linear approximation reasonably valid. As evidenced by the results in Table 12 (detailed in Appendix B.4), this assumption leads to improved robustness in practice. We leave more fine-grained modeling of $\delta\boldsymbol{x}$ to future work.

Thus, we assume that there exists a perturbation coefficient λ such that $\boldsymbol{\delta x} = \lambda\boldsymbol{x}$ (a theoretical analysis with experimental support results detailed in Appendix C. Substituting this into Eq. 8 and eliminating the constant coefficient term, we can obtain:

$$\begin{aligned}\delta E &= \lambda y\boldsymbol{w}^T\boldsymbol{x} + \left(\frac{1}{2}\lambda^2 - \lambda\right)\boldsymbol{x}^T\boldsymbol{w}\boldsymbol{w}^T\boldsymbol{x} + \frac{1}{2}\delta\boldsymbol{w}^T\boldsymbol{H}_{11}\delta\boldsymbol{w} \\ &= \lambda_1 y\boldsymbol{w}^T\boldsymbol{x} + \lambda_2\boldsymbol{x}^T\boldsymbol{H}_{22}\boldsymbol{x} + \frac{1}{2}\delta\boldsymbol{w}^T\boldsymbol{H}_{11}\delta\boldsymbol{w},\end{aligned} \tag{9}$$

where $\lambda_1 = \lambda$, $\lambda_2 = \frac{1}{2}\lambda^2 - \lambda$ denote the perturbation coefficients of the first-order activation bias and the second-order activation bias term, respectively. the goal of pruning w_q should satisfy $w_q + \delta w_q = 0$, i.e., $\boldsymbol{e}_q^T\delta\boldsymbol{w} + \boldsymbol{w} = 0$, the solution of $\delta\boldsymbol{w}$ and δE can be obtained using the Lagrange

multiplier method as follows:

$$\delta \boldsymbol{w} = -\frac{w_q}{\left(\boldsymbol{H}_{11}^{-1}\right)_{qq}} \left(\boldsymbol{H}_{11}^{-1}\right)_{:,q},$$

$$L_q = \lambda_1 y \boldsymbol{w}^T \boldsymbol{x} + \lambda_2 \boldsymbol{x}^T \boldsymbol{H}_{22} \boldsymbol{x} + \frac{1}{2} \frac{w_q^2}{\left(\boldsymbol{H}_{11}^{-1}\right)_{qq}}. \quad (10)$$

Finally, we use L_q as the pruning metric S to select the pruning mask and update the weights with $\delta \boldsymbol{w}$. It is worth noting that, unlike pruning methods such as SparseGPT, Wanda, and Pruner-Zero, our pruning metric S explicitly incorporates the output activation (i.e., y, the input activation of the next layer), demonstrating its ability to model cross-layer dependency.

Attention Distribution-Aware Dynamic Weight Update Strategy

For the q/k/v weights in the attention module, under the pruning mask obtained by dual error minimization and after pruning a certain weight w_m, the weights are updated as $w_i = w_0^i - \delta w_i \, (i \neq m)$, where w_0^i denotes the original i-th weight, $\delta w_i = \alpha_i \, w_q (\alpha_i = (H^{-1})_{qi} / (H^{-1})_{qq})$. The original attention of the i-th token is:

$$\text{Attention}\,(Q, K, V)_i = \sum_{j=1}^{n} \text{soft}\max\left(\frac{\boldsymbol{q}_i \boldsymbol{k}_j^T}{\sqrt{d}}\right) \boldsymbol{v}_j, \quad (11)$$

where $(Q, K, V)_i$ corresponds to the query, key, and value of the i-th token. n denotes the number of tokens and d is the feature dimension. The softmax operator computation result is proportional to the product of $\boldsymbol{q}_i$ and the transposed $\boldsymbol{k}_j$ and the attention is proportional to the elements of $\boldsymbol{v}_j$. The long-tail distribution of multi-head attention is critical for semantic understanding in LLMs, where a small fraction of positions carry disproportionately large, critical weights. Unconstrained linear updates to the remaining q/k/v weights shift these relative magnitudes, distorting the original long-tail pattern and undermining attention fidelity. As illustrated in Figure 3c, the non-weight-update method can better maintain the long-tail distribution while achieving worse model performance. However, by entirely refraining from updating pruned q/k/v weights, local errors propagate and accumulate across layers. To address this issue, we propose an attention distribution-aware dynamic weight update strategy that explicitly preserves key attention patterns via a distribution-constrained optimization objective. After pruning off the m-th weight for q, k,v weights of multi-head attention (MHA) and under the pruning mask selection based on the dual error minimization method, the restoration of q/k/v weights is equivalent to solving the following optimization problems:

$$\begin{aligned} &\min_{\delta w_{q/k/v}} \mathcal{L}_{q/k/v} + \rho \cdot \text{KL}\left(\text{MHA}(\tilde{w}_{q/k/v}) \,||\, \text{MHA}(w_{q/k/v})\right), \\ &\text{s.t.} \quad \delta w_{q/k/v} \in \{0, -\alpha_{q/k/v} w_{q/k/v}^m\}, \\ &\qquad \mathcal{L}_{q/k/v} = \sum_{*\in\{q,k,v\}} \|w_* x_* - (M_* \odot \tilde{w}_*) x_*\|_2^2, \\ &\qquad \tilde{w}_{q/k/v} = w_{q/k/v} + \delta w_{q/k/v}, \end{aligned} \quad (12)$$

where $L_{q/k/v}$ measures the output error from pruning for q, k, and v layer. KL is the Kullback–Leibler divergence enforcing the pruned attention distribution to mimic the original long-tail pattern. ρ is the Lagrangian multiplier, which balances error minimization and distribution preservation. $\delta \boldsymbol{w}_{q/k/v}$ represents $\sigma \boldsymbol{w}_q$, $\sigma \boldsymbol{w}_k$ or $\sigma \boldsymbol{w}_v$, $\alpha_{q/k/v}$ denotes α_q, α_k or α_v. The essence of the above optimization problem is to determine whether the q, k, and v weights need to be updated.

To solve this problem, we explicitly formulate the update strategy of q, k, and v weights as a constrained optimization problem that balances pruning error minimization and attention distribution preservation. Specifically, we design the search space using the binary update status $\{\text{update}, \text{non-update}\}$ for each of the q, k, and v weights in attention layers: "update" enables weight adjustment to compensate for pruning-induced errors, while "non-update" preserves the original weight structure to maintain the long-tail attention distribution. To identify the optimal configuration, we adopt perplexity (ppl) as the objective metric, which comprehensively reflects both the model's language modeling performance (linked to pruning error $L_{q/k/v}$) and the fidelity of attention patterns (linked to KL divergence of distributions). This dynamic adaptation process leverages the cross-layer consistency of outlier ratios in $q/k/v$ weights (e.g., 98% layers in LLaMA-2-13B show the highest outlier ratio in k matrix, see Figure 10 in Appendix D) to reduce the search space, ensuring efficiency while aligning with the dual optimization goal of minimizing error and preserving distribution.

Experiments

Experimental Settings

Implementation details. We compare D^2Prune with three main baselines, including weight-update pruning methods such as SparseGPT (Frantar and Alistarh 2023) and non-weight-update pruning methods such as Wanda (Sun et al. 2023) and Pruner-Zero (Dong et al. 2024), and all experiments are conducted on an NVIDIA A40 GPU. Moreover, given the potential of our dynamic attention update mechanism for global modeling, we also compare our method with a global pruning approach, SparseLLM (Bai et al. 2024), which iteratively updates activations and weights in the MLP modules to minimize global error detailed results in the Appendix B.5). . In addition, we further evaluate our method on the latest large language models Qwen3-8B and 14B (Bai et al. 2023), as well as the vision transformer model DeiT (Touvron et al. 2021) built on the ViT (Dosovitskiy et al. 2020) architecture detailed in Appendix B.3). Following prior studies, we assess and compare various pruning methods based on language modeling with the perplexity metric (ppl) and the zero-shot accuracy for downstream tasks. For all pruning algorithms, we use the C4 (Raffel et al. 2020) training set as the calibration dataset, and we use 128 samples and segment the pre-trained LLMs according to their maximum embedding dimensions. For the language modeling comparison, we assess the perplexity on the WikiText2 (Merity et al. 2016) test set. For the zero-shot comparison, we use seven benchmark tasks including BoolQ (Clark et al. 2019), Hel-

Sparsity	Method	OPT-125M		LLaMA-2-7B		LLaMA-2-13B		LLaMA-2-70B		LLaMA-3-8B	
		PPL ↓	ACC ↑	PPL ↓	ACC ↑	PPL ↓	ACC ↑	PPL ↓	ACC ↑	PPL ↓	ACC ↑
0	Dense	27.66	39.68	5.12	64.38	4.57	67.06	3.12	71.50	5.54	68.40
50	SparseGPT	36.85	39.82	6.52	60.35	5.63	64.87	3.98	71.35	8.56	63.13
	Wanda	38.88	39.75	6.44	60.46	5.58	64.17	3.98	70.96	9.06	61.02
	Pruner-Zero	38.80	39.09	6.43	59.26	5.57	64.21	-	-	8.52	60.28
	D^2Prune **(Ours)**	**34.98**	**40.39**	**6.36**	**61.06**	**5.53**	**65.90**	**3.93**	**71.60**	**8.34**	**63.58**
60	SparseGPT	59.46	39.37	9.56	55.34	7.77	60.26	4.98	70.09	14.40	55.39
	Wanda	74.39	39.45	9.89	53.91	7.87	59.57	4.99	69.04	22.80	48.12
	Pruner-Zero	68.07	39.32	10.34	52.52	7.82	58.12	-	-	20.30	50.65
	D^2Prune **(Ours)**	**52.10**	**40.14**	**9.05**	**56.08**	**7.49**	**60.81**	4.88	**70.65**	**13.44**	**56.95**
70	SparseGPT	218.29	36.31	29.62	44.75	18.20	48.12	8.61	63.49	38.85	43.56
	Wanda	347.42	35.43	84.92	36.68	44.86	39.75	40.27	60.44	114.30	37.07
	Pruner-Zero	317.87	36.88	151.92	38.09	44.76	43.33	-	-	280.33	36.17
	D^2Prune **(Ours)**	**160.81**	**38.09**	**21.10**	**47.97**	**16.51**	**48.32**	**8.17**	**64.06**	**33.37**	**44.82**
80	SparseGPT	2140.55	35.08	102.43	36.23	99.14	38.28	25.86	47.71	178.01	36.95
	Wanda	1920.63	34.93	5107.20	33.72	1384.40	34.67	156.68	37.98	2245.91	34.87
	Pruner-Zero	1251.38	35.06	10244.70	34.83	2040.65	35.09	-	-	10420.01	35.31
	D^2Prune **(Ours)**	**1038.87**	**36.29**	**92.68**	**39.09**	**76.80**	**39.42**	**21.37**	**48.09**	**151.47**	**38.73**
2:4	SparseGPT	59.76	-	10.18	-	8.39	-	5.32	-	14.16	-
	Wanda	79.80		11.35		8.37		5.18		22.86	
	Pruner-Zero	70.92		11.16		8.05		-		23.56	
	D^2Prune **(Ours)**	**59.43**		**10.00**		**8.02**		**5.12**		**14.10**	
3:4	SparseGPT	1365.58	-	154.23	-	147.23	-	54.84	-	281.74	-
	Wanda	2497.68		3111.14		5815.71		386.57		13054.17	
	Pruner-Zero	2946.15		7913.18		4134.56		-		854085.30	
	D^2Prune **(Ours)**	**1346.36**		**136.89**		**143.69**		**48.06**		**190.35**	

Table 1: Pruning comparison across language modeling and zero-shot tasks at different sparsity levels (middle: 50% and 60%, high: 70% and 80%). For language modeling, we evaluate pruning performance by measuring perplexity (PPL) on WikiText2, where a lower perplexity indicates better language modeling capability. For zero-shot tasks, we report the average accuracy (ACC) across seven downstream benchmarks, where a higher accuracy reflects better generalization performance after pruning. 2:4 (middle sparsity level) and 3:4 (high sparsity level) are both N:M pruning (N zero values among M consecutive weights).

laSwag (Zellers et al. 2019), WinoGrande, RTE (Wang et al. 2018), ARC-c and ARC-e (ARC Easy and Challeng (Clark et al. 2018)), and OBQA (Mihaylov et al. 2018) from the EleutherAI LM Harness (Gao et al. 2023). All the datasets are sourced from the HuggingFace Datasets library.

Experimental Results

Unified Evaluation on Language Modeling and Zero-Shot. Following SparseGPT, Wanda and Pruner-Zero, we provide a unified pruning performance evaluation result in Table 1 on both language modeling (in terms of perplexity, PPL↓) and zero-shot tasks (measured by average accuracy, ACC↑) under various sparsity levels and model sizes. Overall, D^2Prune consistently outperforms existing pruning baselines across both moderate (50%–60%, 2:4) and high sparsity (70%–80%, 3:4) regimes. Compared to weight-update methods (e.g., SparseGPT), D^2Prune achieves on average a 3.1% accuracy gain and 16% lower perplexity. Against non-weight-update methods (e.g., Wanda, Pruner-Zero), the perplexity reduction reaches up to 86%. Notably, D^2Prune surpasses the dense model on LLaMA-2-70B at 50% sparsity in zero-shot accuracy, demonstrating strong generalization. Due to resource limitations, Pruner-Zero fails to perform pruning on LLaMA-2-70B even with a single A40 or A100 GPU. Therefore, the corresponding entries are marked as "–" in the table. For semi-structured pruning (e.g., 2:4 and 3:4 patterns), we report only the perplexity results, while accuracy is marked as "–" to indicate that it was not evaluated. The detailed zero-shot results on seven downstream tasks are shown in Appendix B.1, providing a comprehensive understanding of the zero-shot capabilities for the discussed models.

Expanding Experiment Results. (1) We further compare with the more computationally intensive **global pruning** method SparseLLM (Bai et al. 2024) (detailed in Appendix B.5 and (2) validate D^2Prune on **vision transformers** (e.g., DeiT (Touvron et al. 2021)) and **recent LLMs like Qwen3 (Bai et al. 2023)** (detailed in Appendix B.3, confirming its broad applicability across architectures and tasks. (3) In addition, to comprehensively evaluate the reasoning capabilities of pruned models in real-world scenarios, we conduct **in-context learning (ICL) experiments on the GSM8K** benchmark(Cobbe et al. 2021) (detailed in Appendix B.2). The results demonstrate that in the few-shot setting, D^2Prune consistently outperforms baselines. (3) Finally, we provide **pruning efficiency and speedup** experiment results in the Appendix E. Notably, our proposed method achieves better pruning performance than weight-update approaches like SparseGPT without introducing additional computational overhead. Compared to non-weight-update methods, our approach attains significantly higher

accuracy under high sparsity with minor overhead.

Ablation study

Effectiveness of the Dual Taylor Expansion for mask selection & weight update. To validate the effectiveness of the Dual Taylor Expansion for mask selection & weight update in Sec. . We incorporated the first and second order activation partial derivatives to SparseGPT and Wanda for mask selection & weight update, and we denote these methods with the prefix "D^2-". As shown in Table 2, D^2-SparseGPT consistently outperforms SparseGPT across all models and sparsity levels, and D^2-Wanda consistently outperforms Wanda, demonstrating the effectiveness of the Dual Taylor Expansion. Besides, we perform the ablation study on key hyperparameters λ_1, λ_2 and s for Dual Taylor Expansion in Appendix B.4.

OPT-125M Dense (WikiText2: 27.66)					LLaMA-2-7B Dense (WikiText2: 5.12)				
method	Sparsity				method	Sparsity			
	50	60	70	80		50	60	70	80
SparseGPT	36.85	59.46	218.29	2140.55	SparseGPT	6.52	9.56	29.62	102.43
D^2-SparseGPT	**36.03**	**55.39**	**202.50**	**1997.06**	D^2-SparseGPT	**6.46**	**9.46**	**23.45**	**101.88**
Wanda	38.88	74.39	347.42	1920.63	Wanda	6.44	9.89	84.92	5107.20
D^2-Wanda	**38.15**	**71.55**	**332.98**	**1790.44**	D^2-Wanda	**6.43**	**9.75**	**81.62**	**3561.51**
LLaMA-3-8B Dense (WikiText2: 5.54)					**LLaMA-2-13B Dense (WikiText2: 4.57)**				
method	Sparsity				method	Sparsity			
	50	60	70	80		50	60	70	80
SparseGPT	8.56	14.40	38.85	178.01	SparseGPT	5.63	7.77	18.20	99.14
D^2-SparseGPT	**8.48**	**13.66**	**38.64**	**170.92**	D^2-SparseGPT	**5.58**	**7.58**	**17.00**	**82.75**
Wanda	9.06	22.80	114.30	2245.91	Wanda	5.58	7.87	44.86	1384.40
D^2-Wanda	**8.92**	**22.54**	**106.58**	**1414.32**	D^2-Wanda	**5.53**	**7.66**	**41.32**	**1001.00**

Table 2: Ablations study of Dual Taylor Expansion on WikiText2 (in perplexity). "D^2-" indicates that the first and second-order activation partial derivative term S_X (see Algorithm 1 that is detailed in Appendix A).

Effectiveness of Attention Distribution-Aware Dynamic Weight Update Strategy. As shown in Table 3, we perform unstructured pruning on the WikiText2 dataset. Specifically, when either the q, k, or v weight is not updated (denoted as w/o q, w/o k, and w/o v), the pruning performance outperforms Wanda and is competitive with SparseGPT. Furthermore, our attention distribution-aware dynamic weight update strategy dynamically sets one of the q, k, and v weights to the non-update state while keeping the others in the update state, leading to optimal performance for all LLMs. This strongly supports the effectiveness of the attention distribution-aware dynamic weight update strategy. Detailed comparison and analysis are in Table 3 and Appendix D.

Method	Weight-Update Layers	OPT	LLaMA-2			LLaMA-3
		125M	7B	13B	70B	8B
SparseGPT	ALL Layers	2140.55	102.43	99.14	25.86	178.01
Wanda	None	1920.63	5107.20	1384.40	156.68	2245.91
Pruner-Zero	None	1251.38	10244.70	2040.65	-	10420.01
D^2Prune	ALL Layers	2469.78	101.88	82.75	24.86	170.92
	None	1790.44	3561.51	1001.00	150.94	1414.32
	w/o q	2077.36	104.63	89.19	24.59	230.83
	w/o k	1038.87	118.79	76.80	25.61	190.48
	w/o v	2410.76	92.68	83.58	21.37	151.47
	Dynamic Update	**1038.87**	**92.68**	**76.80**	**21.37**	**151.47**

Table 3: Ablations study of dynamic weight update for attention modules (q, k, v) on WikiText2 (in perplexity, unstructured 80% sparsity).

Analysis

Impact of sparsity on the update configurations of q, k, v weights. When searching for optimal q, k, and v update configurations, we evaluated various settings under different sparsity levels. The configuration yielding the lowest perplexity was selected as the optimal setup. The results in Table 3 and Appendix D reveal a key insight: the optimal q, k, and v update configuration remains consistent across all sparsity levels, indicating that it is independent of sparsity.

LoRA Fine-tuning. We evaluated the impact of fine-tuning on the pruned models. Specifically, we employ LoRA (Hu et al. 2022) (r=8, α=16) for fine-tuning with one epoch on the C4 training datasets (training sample size is 30k) using one GPU (40G memory) and 15 hours. We conducted experiments on LLaMA-2-7B, involving unstructured pruning at 60% and 70% sparsity levels and semi-structured pruning (2:4). Table 4 reports the perplexity on the WikiText2 dataset and mean accuracy of zero-shot tasks. We can observe that LoRA fine-tuning can effectively restore the performance of all pruned models, and D^2Prune consistently outperforms all other pruning methods in both accuracy and perplexity. This further demonstrates the effectiveness of D^2Prune.

Evaluation	Dense	Method	Fine-tuning	60%	70%	2:4
Zero-Shot	64.38	SparseGPT	✗	55.34	44.75	55.29
			LoRA	57.82	49.73	56.76
		Wanda	✗	53.91	36.68	52.18
			LoRA	56.45	46.43	55.86
		D^2Prune	✗	**56.08**	**47.97**	**55.76**
			LoRA	**60.59**	**51.37**	**57.51**
Perplexity	5.12	SparseGPT	✗	9.56	29.62	10.18
			LoRA	7.18	10.18	7.26
		Wanda	✗	9.86	84.92	11.20
			LoRA	7.26	11.81	7.41
		D^2Prune	✗	**9.05**	**21.10**	**10.00**
			LoRA	**7.05**	**9.70**	**7.21**

Table 4: Fine-tuning can mitigate the perplexity gap to dense LLM. D^2Prune significantly outperforms all other pruning approaches even after LoRA fine-tuning.

Conclusion

Our work proposed D^2Prune, a novel pruning algorithm for compressing large language models (LLMs). By enhancing existing error minimization methods with Dual Taylor Expansion and attention distribution awareness, D^2Prune improved the accuracy of pruning mask selection and weight updates. It accounted for the dual impact of activation and weight variations within error minimization. Besides, it incorporated a dynamic weight update strategy for attention modules in Transformer architectures, reducing computational errors in attention distribution. Experimental results demonstrate that D^2Prune achieved substantial performance improvements under various sparsity conditions, obtaining superior performance than recent post-training pruning methods. Limitations and future works are detailed in Appendix F[1].

[1]Supplementary materials are available at https://github.com/cquxl/D2Prune/.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Nos. 62572085 and 62472058).

References

Bai, G.; Li, Y.; Ling, C.; Kim, K.; and Zhao, L. 2024. SparseLLM: Towards global pruning of pre-trained language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Bai, J.; Bai, S.; Chu, Y.; Cui, Z.; Dang, K.; Deng, X.; Fan, Y.; Ge, W.; Han, Y.; Huang, F.; Hui, B.; Ji, L.; Li, M.; Lin, J.; Lin, R.; Liu, D.; Liu, G.; Lu, C.; Lu, K.; Ma, J.; Men, R.; Ren, X.; Ren, X.; Tan, C.; Tan, S.; Tu, J.; Wang, P.; Wang, S.; Wang, W.; Wu, S.; Xu, B.; Xu, J.; Yang, A.; Yang, H.; Yang, J.; Yang, S.; Yao, Y.; Yu, B.; Yuan, H.; Yuan, Z.; Zhang, J.; Zhang, X.; Zhang, Y.; Zhang, Z.; Zhou, C.; Zhou, J.; Zhou, X.; and Zhu, T. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609*.

Blalock, D.; Gonzalez Ortiz, J. J.; Frankle, J.; and Guttag, J. 2020. What is the state of neural network pruning? *Proceedings of machine learning and systems*, 2: 129–146.

Boža, V. 2024. Fast and effective weight update for pruned large language models. *arXiv preprint arXiv:2401.02938*.

Chen, J.; Agarwal, A.; Abdelkarim, S.; Zhu, D.; and Elhoseiny, M. 2022. Reltransformer: A transformer-based long-tail visual relationship recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 19507–19517.

Chi, H.; Li, H.; Yang, W.; Liu, F.; Lan, L.; Ren, X.; Liu, T.; and Han, B. 2024. Unveiling causal reasoning in large language models: Reality or mirage? *Advances in Neural Information Processing Systems*, 37: 96640–96670.

Chijiwa, D.; Yamaguchi, S.; Ida, Y.; Umakoshi, K.; and Inoue, T. 2021. Pruning randomly initialized neural networks with iterative randomization. *Advances in neural information processing systems*, 34: 4503–4513.

Clark, C.; Lee, K.; Chang, M.-W.; Kwiatkowski, T.; Collins, M.; and Toutanova, K. 2019. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*.

Clark, P.; Cowhey, I.; Etzioni, O.; Khot, T.; Sabharwal, A.; Schoenick, C.; and Tafjord, O. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Dong, P.; Li, L.; Tang, Z.; Liu, X.; Pan, X.; Wang, Q.; and Chu, X. 2024. Pruner-Zero: Evolving Symbolic Pruning Metric from scratch for Large Language Models. *arXiv preprint arXiv:2406.02924*.

Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.

Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Frantar, E.; and Alistarh, D. 2023. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*, 10323–10337. PMLR.

Gao, L.; Tow, J.; Abbasi, B.; Biderman, S.; Black, S.; DiPofi, A.; Foster, C.; Golding, L.; Hsu, J.; Le Noac'h, A.; et al. 2023. A framework for few-shot language model evaluation, 12 2023. *URL https://zenodo. org/records/10256836*, 7.

Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; Bi, X.; et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*.

Han, S.; Pool, J.; Tran, J.; and Dally, W. 2015. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28.

Hassibi, B.; Stork, D. G.; and Wolff, G. J. 1993. Optimal brain surgeon and general network pruning. In *IEEE international conference on neural networks*, 293–299. IEEE.

Hoang, D. N.; and Liu, S. 2023. Revisiting pruning at initialization through the lens of ramanujan graph. *ICLR 2023*.

Hu, E. J.; yelong shen; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

Ji, T.; Jain, S.; Ferdman, M.; Milder, P.; Schwartz, H. A.; and Balasubramanian, N. 2021. On the distribution, sparsity, and inference-time quantization of attention values in transformers. *arXiv preprint arXiv:2106.01335*.

Kim, B.-K.; Kim, G.; Kim, T.-H.; Castells, T.; Choi, S.; Shin, J.; and Song, H.-K. 2024. Shortened llama: A simple depth pruning for large language models. *arXiv preprint arXiv:2402.02834*, 11.

Kwon, W.; Kim, S.; Mahoney, M. W.; Hassoun, J.; Keutzer, K.; and Gholami, A. 2022. A fast post-training pruning framework for transformers. *Advances in Neural Information Processing Systems*, 35: 24101–24116.

LeCun, Y.; Denker, J.; and Solla, S. 1989. Optimal Brain Damage. In Touretzky, D., ed., *Advances in Neural Information Processing Systems*, volume 2. Morgan-Kaufmann.

Li, J.; Tang, T.; Zhao, W. X.; Nie, J.-Y.; and Wen, J.-R. 2024a. Pre-trained language models for text generation: A survey. *ACM Computing Surveys*, 56(9): 1–39.

Li, L.; Dong, P.; Tang, Z.; Liu, X.; Wang, Q.; Luo, W.; Xue, W.; Liu, Q.; Chu, X.; and Guo, Y. 2024b. Discovering sparsity allocation for layer-wise pruning of large language models. *Advances in Neural Information Processing Systems*, 37: 141292–141317.

Liu, Z.; Sun, M.; Zhou, T.; Huang, G.; and Darrell, T. 2018. Rethinking the value of network pruning. *arXiv preprint arXiv:1810.05270*.

Merity, S.; Xiong, C.; Bradbury, J.; and Socher, R. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*.

Mihaylov, T.; Clark, P.; Khot, T.; and Sabharwal, A. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*.

Mo, Y.; Qin, H.; Dong, Y.; Zhu, Z.; and Li, Z. 2024. Large language model (llm) ai text generation detection based on transformer deep learning algorithm. *arXiv preprint arXiv:2405.06652*.

OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774.

Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; and Liu, P. J. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140): 1–67.

Reyhan, Z. A.; Rahnamayan, S.; and Bidgoli, A. A. 2024. Novel Post-Training Structure-Agnostic Weight Pruning Technique for Deep Neural Networks. In *2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2206–2212. IEEE.

Sakaguchi, K.; Bras, R. L.; Bhagavatula, C.; and Choi, Y. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9): 99–106.

Shao, W.; Chen, M.; Zhang, Z.; Xu, P.; Zhao, L.; Li, Z.; Zhang, K.; Gao, P.; Qiao, Y.; and Luo, P. 2023. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137*.

Sreenivasan, K.; Sohn, J.-y.; Yang, L.; Grinde, M.; Nagle, A.; Wang, H.; Xing, E.; Lee, K.; and Papailiopoulos, D. 2022. Rare gems: Finding lottery tickets at initialization. *Advances in neural information processing systems*, 35: 14529–14540.

Sun, M.; Liu, Z.; Bair, A.; and Kolter, J. Z. 2023. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*.

Tanaka, H.; Kunin, D.; Yamins, D. L.; and Ganguli, S. 2020. Pruning neural networks without any data by iteratively conserving synaptic flow. *NeurIPS*, 33: 6377–6389.

Touvron, H.; Cord, M.; Douze, M.; Massa, F.; Sablayrolles, A.; and Jégou, H. 2021. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, 10347–10357. PMLR.

van der Ouderaa, T. F.; Nagel, M.; Van Baalen, M.; Asano, Y. M.; and Blankevoort, T. 2023. The llm surgeon. *arXiv preprint arXiv:2312.17244*.

Wang, A.; Singh, A.; Michael, J.; Hill, F.; Levy, O.; and Bowman, S. R. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*.

Wang, Z. 2024. CausalBench: A Comprehensive Benchmark for Evaluating Causal Reasoning Capabilities of Large Language Models. In *Proceedings of the 10th SIGHAN Workshop on Chinese Language Processing (SIGHAN-10)*, 143–151.

Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; and Han, S. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, 38087–38099. PMLR.

Zellers, R.; Holtzman, A.; Bisk, Y.; Farhadi, A.; and Choi, Y. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*.

Zhang, Y.; Bai, H.; Lin, H.; Zhao, J.; Hou, L.; and Cannistraci, C. V. 2024. Plug-and-play: An efficient post-training pruning method for large language models. In *The Twelfth International Conference on Learning Representations*.

Zhou, H.; Zhang, S.; Peng, J.; Zhang, S.; Li, J.; Xiong, H.; and Zhang, W. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, 11106–11115.

Algorithm 1: D^2Prune: Dynamic Dual-layer Pruning with Selective Q/K/V Update. We prune all the layer weight matrix of FFN or MLP by weight-update method (SparseGPT (Frantar and Alistarh 2023) solver based on Dual Taylor Expansion), and prune the layer weight matrix W_q, W_k or W_v of multi-head attention (MHA) by non-weight-update method (Wanda (Sun et al. 2023) solver based on Dual Taylor Expansion) detailed in Appendix D). We apply the scaling factor s to both input and output activations in the activation derivative terms to standardize and balance the weight magnitudes (detailed in Appendix B.4). For example, $||a||_2 \leftarrow s\left(||a||_2\right) = \left(||a||_2^2/s\right)^{1/2}$ for activations a.

Input: Transformer layer with MHA and FFN/MLP modules. For FFN/MLP: weight matrix $FC1/W_{\text{up}}$,$\text{FC2}/W_{\text{down}}$. For MHA: weight matrix W_q, W_k, W_v, W_o, input activations X, output activations Y, target sparsity $p\%$, hyperparameters λ_1, λ_2, scaling factor s.

Function D^2Prune **on FFN** (W_{up}, W_{down}, X, Y)**:**

Apply scaling: $\|X\|_2 \leftarrow \left(\|X\|_2^2/s\right)^{1/2}$, $\|Y\|_2 \leftarrow \left(\|Y\|_2^2/s\right)^{1/2}$

foreach *linear layer* $l \in \{up, down\}$ **do**

`// Prune by weight-update method ← SparseGPT Solver based on Dual Taylor expansion`

Compute mask M_l selecting $(1-p\%)$ weights $w_q \in W_l$ with highest scores:

$$\lambda_1\|Y\|_2 \cdot |W| \cdot \|X\|_2 + \lambda_2|W|^2 \cdot \|X\|_2^2 + \frac{1}{2}\frac{w_q^2}{(H_{11}^{-1})_{qq}}$$

Pruning weights: $W_l \leftarrow (1-M_l) \cdot W_l$

Update weights: $W_l \leftarrow W_l - \frac{w_q}{(H_{11}^{-1})_{qq}} \cdot (H_{11}^{-1})_{:,q}$

return $W_{\text{up}}, W_{\text{down}}$

Function D^2Prune **on MHA** $(W_q, W_k, W_v, W_o, X, Y)$**:**

foreach $l_{non} \in \{q, k, v\}$ **do**

foreach $l \in \{q, k, v, o\}$ **do**

if $l == l_{non}$ **then**

`// Prune by non-weight-update method ← Wanda Solver based on Dual Taylor expansion`

Compute mask M_l selecting $(1-p\%)$ weights $w_q \in W_l$ with highest scores:

$$\lambda_1\|Y\|_2 \cdot |W| \cdot \|X\|_2 + \lambda_2|W|^2 \cdot \|X\|_2^2 + |W|^2 \cdot \|X\|_2^2$$

Pruning weights: $W_l \leftarrow (1-M_l) \cdot W_l$

else

`// Prune by weight-update method ← SparseGPT Solver based on Dual Taylor expansion`

Compute mask M_l selecting $(1-p\%)$ weights $w_q \in W_l$ with highest scores:

$$\lambda_1\|Y\|_2 \cdot |W| \cdot \|X\|_2 + \lambda_2|W|^2 \cdot \|X\|_2^2 + \frac{1}{2}\frac{w_q^2}{(H_{11}^{-1})_{qq}}$$

Pruning weights: $W_l \leftarrow (1-M_l) \cdot W_l$

Update weights: $W_l \leftarrow W_l - \frac{w_q}{(H_{11}^{-1})_{qq}} \cdot (H_{11}^{-1})_{:,q}$

Apply pruned $\{W_q, W_k, W_v\}$ to MHA module and evaluate model performance (e.g., PPL).

Select arg min-PPL combination of $\{q, k, v\}$ as l^*_{non}.

return Optimal $\{W_q, W_k, W_v\}$ under l^*_{non}, W_o

A D^2Prune Algorithm Pseudo-Code

In Section 3 (detailed in main text), we provide the detailed theoretical derivation of D^2Prune. We can unify the weight importance function S for pruning mask selection into the following formula:

$$S = f\left(X, Y, W\right) = S_X + S_W, \tag{13}$$

where S_X is the first and second-order partial derivative term of the error with respect to activations, and S_W is the second-order partial derivative term of the error with respect to weights, satisfying:

$$S_X = \lambda_1 YWX + \lambda_2 X^T H_{22} X, S_W = \frac{1}{2}\frac{W^2}{H_{11}^{-1}}. \tag{14}$$

Note that $H_{22} = WW^T$ and $H_{11} = XX^T$. To avoid the complex matrix inversion required for computing the Hessian and to accelerate the calculation of D^2Prune, we simplify S_W in the non-weight-update pruning method by following the idea proposed in Wanda(Sun et al. 2023), resulting in $S_W = \frac{1}{2}W^2X^2$. Additionally, inspired by quantization methods such as SmoothQuant (Xiao et al. 2023) and OmniQuant (Shao et al. 2023), we introduce scaling factors to balance the magnitudes of weights and activations. This scaling operation is crucial to appropriately reflect the importance of both components during pruning without compromising pruning accuracy. Such that:

$$\begin{aligned} S_X &= \lambda_1 s_x^1(Y) \cdot s_x^2(W) \cdot s_x^1(X) + \lambda_2 s_x^2(W) \cdot s_x^1(X), \\ S_W &= s_w^2(W) \cdot s_w^1(X). \end{aligned} \tag{15}$$

where $s_x = \left(s_x^1, s_x^2\right)$ and $s_w = \left(s_w^1, s_w^2\right)$ are the scaling factors for the first-order activation partial derivative term and the second-order weight partial derivative term, respectively. Specifically, s_a^1 is regularized using the L2 norm and scaled by a certain magnitude s ($s = 1500$ in this paper for all the experiments) to prevent excessive dominance due to scaling differences, while s_a^2 is regularized using the L1 norm to measure the relative magnitude of the weights. In Algorithm 1, we provide the detailed pseudocode for D^2Prune.

B Expanding the Experimental Results

B.1 Expanding the Zero-Shot Taks

Detailed Results. Table 5 reports the detailed zero-shot results for OPT-125M, LLaMA-2-7B, LLaMA-2-13B, LLaMA-2-70B, and LLaMA-3-8B across 7 downstream tasks including BoolQ (Clark et al. 2019), HellaSwag (Zellers et al. 2019), WinoGrande, RTE (Wang et al. 2018), ARC-c and ArC-e (ARC Easy and Challeng (Clark et al. 2018)), and OBQA (Mihaylov et al. 2018), respectively, providing a comprehensive understanding of the zero-shot capabilities for the discussed models. For a fair comparison, all methods are kept with the same setup, and the evaluation tools are consistent, with the same lm_eval version (v0.4.4). Since the pruning results for LLaMA-2-70B with Pruner-Zero could not be reproduced on a single GPU (NVIDIA A100).

Analysis and Discussions. Overall, for the zero-shot accuracy, it can be seen that although there are certain differences across tasks, D^2Prune significantly outperforms non-weight-update pruning methods such as Wanda and Pruner-Zero in most tasks under medium and high sparsity conditions. Compared to weight-update methods, under the medium sparsity condition, the performance of D^2Prune is close to SparseGPT. However, as sparsity increases, D^2Prune outperforms SparseGPT by a large margin, with performance improvements becoming more noticeable and obvious. This may be due to the significant improvement of D^2Prune in the accuracy of local error estimation at high sparsity compared to other pruning methods. Furthermore, it is worth noting that at certain sparsity levels, the pruned model's performance on downstream tasks can even exceed the Dense baseline. This indicates that pruning involves removing redundant or less important parameters from the model, which can act as a form of regularization or denoising. Similar to dropout, pruning can regularize and eliminate noise in the Dense model, thereby improving generalization capability. This provides strong evidence for the potential of model pruning that offers an effective solution for LLMs compression. For example, As is shown in Table 1 (detailed in main text), under 50% and 60% sparsity conditions of OPT-125M, all pruning algorithms outperform the Dense baseline on the BoolQ dataset, with D^2Prune achieving the best results, surpassing Dense by 12% and 11%, respectively. Additionally, on the RTE dataset, D^2Prune outperforms Dense under both 50% and 60% sparsity, while other pruning algorithms performed slightly worse than Dense, further proving the superiority of our proposed method. Finally, we can also observe that SparseGPT and D^2Prune outperform Dense under 50% and 60% sparsity condition for LLaMA-2-70B pruning.

B.2 Evaluation of In-Context Learning

To comprehensively evaluate the reasoning capabilities of pruned models in real-world scenarios, we conduct in-context learning (ICL) experiments on the GSM8K benchmark. GSM8K(Cobbe et al. 2021) consists of grade-school math word problems that require strong logical reasoning and multi-step arithmetic, making it a challenging testbed for language models. Following prior studies (Sun et al. 2023; Dong et al. 2024), we perform 5-shot unstructured ICL evaluations on LLaMA-2-7B and LLaMA-2-13B. In Table 6, we report the accuracy of the dense LLMs and their pruned counterparts under 60% and 70% sparsity levels. The results demonstrate that in the few-shot setting, D^2Prune consistently outperforms existing pruning methods, including SparseGPT (Frantar and Alistarh 2023), Wanda (Sun et al. 2023), and Pruner-Zero (Dong et al. 2024), highlighting its superior ability to preserve reasoning performance under aggressive compression.

B.3 Additional LLMs and Vision Model Pruning

In this appendix, we explore whether D^2Prune generalizes to other language and vision models. We apply it to the latest Qwen3 (Bai et al. 2023) series of large language models (8B and 14B), as well as the widely used DeiT (Touvron et al. 2021) model based on the ViT architecture. We report perplexity on WikiText2 for Qwen3-8B/14B and top-1 accuracy on ImageNet-1K for DeiT. As shown in Table 7,

Dataset	Method	OPT-125M				LLaMA-2-7B				LLaMA-2-13B				LLaMA-2-70B				LLaMA-3-8B			
		50	60	70	80	50	60	70	80	50	60	70	80	50	60	70	80	50	60	70	80
BoolQ	SparseGPT	61.19	60.42	41.22	39.32	76.52	**73.42**	65.08	45.93	82.05	78.77	*66.81*	60.91	**84.52**	84.22	80.67	69.91	77.06	75.4	**67.52**	47.98
	Wanda	62.11	61.46	39.02	37.88	76.51	66.39	42.29	37.83	81.68	77.15	62.32	37.83	83.12	83.67	74.25	62.20	**79.32**	66.05	49.36	37.98
	Pruner-Zero	61.37	59.35	44.98	38.01	72.84	65.22	45.30	37.83	80.94	77.12	62.72	37.83	-	-	-	-	76.36	68.74	38.13	37.83
	D^2Prune **(Ours)**	**62.17**	**61.59**	**50.40**	**40.89**	**76.61**	73.03	**66.79**	**61.16**	**81.90**	**78.78**	66.79	**62.02**	84.46	**84.86**	**80.83**	**72.54**	79.30	**77.16**	67.22	**56.18**
HellaSwag	SparseGPT	29.77	29.03	**28.47**	26.46	70.51	61.84	40.16	29.60	75.31	**67.57**	**46.52**	29.54	81.52	**78.93**	68.46	**43.65**	71.59	59.63	**37.65**	27.78
	Wanda	30.28	29.21	27.87	26.21	70.70	58.51	29.93	26.63	**76.11**	66.04	30.70	27.10	81.32	77.80	63.80	29.06	67.82	46.50	29.33	27.53
	Pruner-Zero	**30.46**	**29.72**	27.93	26.04	69.52	56.90	29.74	26.18	74.68	65.44	36.27	26.27	-	-	-	-	68.21	51.55	29.98	27.53
	D^2Prune **(Ours)**	30.10	29.42	28.26	**26.90**	**70.88**	**62.05**	**45.81**	**29.81**	76.01	67.48	46.23	**29.81**	**81.54**	78.85	**68.69**	43.18	**72.13**	**60.06**	38.48	**28.25**
WinoGrande	SparseGPT	52.88	52.40	**52.95**	50.59	69.85	65.66	57.14	48.61	71.43	69.85	60.61	48.38	78.05	**78.69**	75.69	**61.88**	72.21	**67.87**	**55.25**	49.72
	Wanda	52.57	52.24	51.38	49.72	68.19	65.11	52.33	**50.67**	70.87	65.50	50.04	50.27	77.51	76.16	73.80	48.22	69.77	59.03	48.54	50.04
	Pruner-Zero	50.35	51.69	51.85	49.32	67.96	62.83	51.14	50.12	70.79	65.67	54.14	**50.59**	-	-	-	-	69.37	60.14	50.43	50.67
	D^2Prune **(Ours)**	**52.89**	**52.96**	52.01	**53.12**	**70.17**	**66.54**	**60.30**	50.30	**72.14**	**70.80**	**61.17**	50.36	**78.37**	78.14	**75.69**	61.72	**72.61**	67.48	54.93	**49.80**
RTE	SparseGPT	49.10	49.09	50.90	52.70	53.43	53.43	54.15	53.07	63.89	56.32	53.06	52.70	70.75	71.84	62.45	57.04	62.81	56.31	52.70	52.70
	Wanda	48.01	49.45	52.34	51.62	53.79	53.42	52.70	45.85	58.12	59.56	52.70	52.70	72.92	69.31	61.01	49.46	59.20	52.70	52.70	52.70
	Pruner-Zero	44.76	48.73	52.70	**53.43**	55.60	53.43	53.07	52.70	62.81	57.40	52.70	52.70	-	-	-	-	55.96	52.71	52.70	52.70
	D^2Prune **(Ours)**	**49.46**	**50.54**	**53.43**	53.07	**55.60**	**54.51**	**54.87**	**53.43**	**66.43**	**61.37**	**53.43**	**52.71**	**72.20**	**74.00**	**64.62**	**57.40**	**64.26**	**61.37**	**54.87**	**52.71**
ARC-c	SparseGPT	23.12	23.21	21.67	21.93	**42.49**	34.72	25.94	22.35	45.13	**42.66**	**27.64**	22.86	55.80	53.15	44.80	**27.47**	**45.13**	34.72	23.80	22.18
	Wanda	22.44	21.58	21.33	23.46	42.40	33.53	22.27	24.14	46.07	40.35	20.95	24.48	55.55	52.13	41.21	21.93	43.51	29.18	22.01	22.61
	Pruner-Zero	22.78	22.69	21.92	22.44	39.51	33.45	22.95	**26.02**	44.11	37.54	24.57	25.34	-	-	-	-	44.88	32.59	21.08	23.55
	D^2Prune **(Ours)**	**24.15**	**23.63**	**21.93**	**23.81**	42.24	**35.58**	**27.30**	25.17	**46.84**	41.13	27.56	**25.17**	**56.31**	**54.01**	**45.48**	26.95	44.20	**36.34**	**24.40**	**24.15**
ARC-e	SparseGPT	37.03	**36.83**	32.95	29.12	68.69	61.11	42.21	28.40	71.04	**65.03**	49.20	28.53	**80.76**	78.58	70.75	44.44	**71.50**	57.36	40.19	**30.47**
	Wanda	36.67	35.18	32.28	28.40	69.06	61.40	30.81	27.52	71.54	64.77	32.70	26.68	78.91	77.57	70.41	28.20	67.71	52.57	30.72	27.61
	Pruner-Zero	37.07	36.82	**33.75**	**29.33**	68.39	59.81	36.45	26.34	71.33	65.23	44.90	27.10	-	-	-	-	68.77	55.81	35.48	28.49
	D^2Prune **(Ours)**	**37.58**	36.41	33.42	29.25	**69.11**	**61.62**	**47.94**	**28.49**	**72.35**	64.90	**49.28**	**29.29**	80.72	**79.29**	**71.34**	**44.44**	69.78	**59.43**	**43.06**	29.84
OBQA	SparseGPT	25.60	24.60	26.00	25.40	41.00	37.20	28.60	25.60	45.20	65.03	49.20	28.53	48.00	45.20	41.60	29.60	41.60	36.40	27.80	27.80
	Wanda	26.20	27.00	23.80	**27.20**	42.60	39.00	26.40	23.40	44.80	64.77	32.70	26.68	47.40	46.60	38.60	26.80	39.80	30.80	26.80	25.60
	Pruner-Zero	**26.80**	26.20	25.00	26.80	41.00	36.00	28.00	**26.18**	44.80	65.23	44.90	27.10	-	-	-	-	38.40	33.00	25.40	26.40
	D^2Prune **(Ours)**	26.40	**26.40**	**27.20**	27.00	**42.80**	**39.20**	**32.80**	25.20	**45.60**	**64.90**	**49.28**	**29.29**	**47.60**	**45.40**	**41.80**	**30.40**	**42.80**	**36.80**	**30.80**	**30.20**
Mean	SparseGPT	39.82	39.37	36.31	35.08	60.35	55.34	44.75	36.23	64.87	60.26	48.12	38.28	71.35	70.09	63.49	47.71	63.13	55.39	43.56	36.95
	Wanda	39.75	39.45	35.43	34.93	60.46	53.91	36.68	33.72	64.17	59.57	39.75	34.67	70.96	69.04	60.44	37.98	61.02	48.12	37.07	34.87
	Pruner-Zero	39.09	39.32	36.88	35.06	59.26	52.52	38.09	34.83	64.21	58.12	43.33	35.09	-	-	-	-	60.28	50.65	36.17	35.31
	D^2Prune **(Ours)**	**40.39**	**40.14**	**38.09**	**36.29**	**61.06**	**56.08**	**47.97**	**39.09**	**65.90**	**60.81**	**48.32**	**39.42**	**71.60**	**70.65**	**64.04**	**48.09**	**63.58**	**56.95**	**44.82**	**38.73**

Table 5: Zero-shot accuracy for each task and their mean accuracy (%) under varying sparsity (50, 60, 70, 80 (%)) for OPT-125M and LLaMA-2/3 Models.

Dataset	GSM8K	
Model	LLaMA-2-7B	LLaMA-2-13B
Dense	13.95	23.28
SparseGPT	2.58	4.17
Wanda	2.20	3.79
Pruner-Zero	1.82	3.03
D^2Prune	**5.84**	**4.47**

Table 6: In-Context Learning Accuracy on the GSM8K Dataset

Evaluation		WikiText: Perplextity		ImageNet: Accuracy
Model		Qwen3-8B	Qwen3-14B	DeiT
Dense		8.56	7.50	87.03
60	SparseGPT	12.72	10.42	86.24
	Wanda	14.49	11.11	83.32
	$D2Prune$	**12.31**	**10.10**	**87.14**
80	SparseGPT	75.20	66.00	72.37
	Wanda	409.34	339.80	28.71
	$D2Prune$	**72.54**	**48.15**	**74.76**

Table 7: Evaluation of $D2Prune$ on Language (Qwen3) and Vision (DeiT) Models

D^2Prune consistently outperforms other pruning methods, further demonstrating its effectiveness and generality.

B.4 Hyperparameter Selection for D^2Prune

Perturbation coefficients: λ_1 and λ_2. First, During the pruning mask selection based on the Dual Taylor Expansion of error, there are hyperparameters λ_1 and λ_2 (in Eq. (10) (detailed in main text) that represent the perturbation coefficients of the first-order activation bias and the second-order activation bias term, respectively. In the range of values, we set the value of λ_1 and λ_2 from 0 to 1 (It is worth noting that in our implementation, a negative sign is applied to λ_2. Therefore, we focus on the relative variation of the coefficients rather than their signs). Specifically, the parameter search space is set to [0, 0.2, 0.5, 0.8, 1], and we perform a study within this range of hyperparameters to explore their impact on pruning performance. Results for OPT-125M, LLaMA-2-7B, LLaMA-2-13B and LLaMA-3-8B with 80% sparsity are shown in Table 8. We can see that none of the hyperparameter experiments achieved the lowest perplexity at (λ_1,λ_2)=(0,0), which further emphasizes the effectiveness of the activation partial derivatives in pruning mask selection. This key result also confirms our hypothesis: since activations are computed from randomly selected mini-batches of calibration data, they inevitably differ from training data in both task type and data distribution. Such discrepancies have a non-negligible impact on layer-wise error estimation during pruning. It is important to note that the theoretical origin of these perturbation coefficients is determined by the differences in activation distributions between the training and calibration datasets, as well as between the calibration and downstream task data. Therefore, by introducing tunable hyperparameters, we aim to guide practitioners in adapting the pruning model based on task distribution discrepancies, which has practical significance.

For instance, when the downstream task closely matches the calibration dataset, we recommend the default setting

of $\lambda = 1$ (i.e., $\lambda_1 = 1$, $\lambda_2 = 0$), where the second-order activation perturbation is small and can be treated as a higher-order infinitesimal. In contrast, when there is a substantial mismatch—such as using language modeling data (e.g., C4 or WikiText2) for calibration but evaluating on more complex tasks involving contextual or commonsense reasoning (e.g., HellaSwag)—adjusting λ_1 and λ_2 can significantly improve inference performance. Our experiments suggest that setting λ_1 and λ_2 to values around 0.5 often yields better results in such cases, as shown in Table 9. We hope this key insight helps readers better understand the practical role of activation derivatives and lays a solid foundation for future research in this direction.

OPT-125M					
λ_1/λ_2	0	0.2	0.5	0.8	1
0	1007.98	992.46	969.23	983.60	970.02
0.2	1007.04	1002.04	**958.73**	1010.90	981.99
0.5	1074.76	1004.22	1002.85	969.67	1029.56
0.8	1096.42	1070.81	1083.9	1046.11	1031.64
1	1038.87	1069.44	983.91	1044.52	1027.64

LLaMA-2-7B					
λ_1/λ_2	0	0.2	0.5	0.8	1
0	98.28	101.39	102.03	100.34	100.20
0.2	99.23	101.27	95.17	97.99	102.43
0.5	102.40	101.47	106.73	98.33	101.56
0.8	101.15	109.74	98.47	99.57	101.74
1	**92.68**	103.81	99.23	103.27	96.47

LLaMA-2-13B					
λ_1/λ_2	0	0.2	0.5	0.8	1
0	89.85	89.94	91.90	93.03	117.55
0.2	88.52	87.84	85.20	88.89	103.4
0.5	84.16	85.94	83.93	82.35	94.95
0.8	89.70	80.80	84.00	85.12	91.66
1	**76.80**	86.04	88.72	86.04	90.61

LLaMA-3-8B					
λ_1/λ_2	0	0.2	0.5	0.8	1
0	161.02	186.67	167.10	167.12	162.42
0.2	141.99	162.85	162.91	179.72	152.58
0.5	**135.43**	160.24	145.03	160.39	149.59
0.8	160.77	153.90	159.70	158.45	160.97
1	151.47	149.73	147.86	146.94	145.70

Table 8: Hyperparameters λ_1 and λ_2 search results of D^2Prune for each model with 80% sparsity and the corresponding perplexity on WikiText2; lower perplexity is better.

Scaling factor: s. In the weight importance evaluation, we introduce a scaling factor s (see Algorithm 1) when computing the activation norm to balance the magnitudes of weights and activations. This serves a role similar to normalization and has been widely adopted across various domains, such as activation scaling in quantization (e.g., SmoothQuant (Xiao et al. 2023), OmniQuant (Shao et al. 2023)) and gradient scaling in pruning (e.g., Pruner-Zero (Dong et al. 2024)). To further investigate the physical meaning of the scaling factor s, we derive its theoretical value based on activation norms. Specifically, we estimate the scaling ratio between x and y by analyzing the relationship between the input activation norm $||x||$ and the output activation norm $||y||$. According to the method of moments and the law of large numbers, we have:

$$\mathbb{E}_{\boldsymbol{x}}\left[\boldsymbol{x}^2\right] \rightarrow \frac{1}{n}\sum_{i=1}^{n}\boldsymbol{x}_i^2 = \frac{||\boldsymbol{x}||_2^2}{n} \tag{16}$$

where, n is the number of samples. It is also the activation length L in this paper.

As shown in eq. 16, the theoretical value of the scaling factor corresponds to the length of the activation vector, which in our case is the sequence length L ($s = L$ can be approximated when L is large). In practice, L is often manually set during preprocessing of the calibration data for model input—e.g., 2048 or 4096—and ideally matches the maximum sequence length of pre-trained model (if have sufficient memory resources in your GPU). In such cases, we recommend setting $s = L$, which can be determined automatically without requiring hyperparameter tuning.

Conversely, when L is small, it is preferable to treat s as a tunable hyperparameter, as it depends on the specific activation distribution. Based on our experiments, setting s in the range of 1000 to 2000 typically yields better pruning performance, as illustrated in Figure 5. Moreover, as shown in Figure 6, pruning models with the scaling factor s consistently outperforms those without it, highlighting the necessity of incorporating the scaling factor.

Sparsity	Method	Hyperparameter		Calibration data: C4	
		λ_1	λ_2	HellaSwag: acc	WikiText2: ppl
50	SparseGPT	-	-	70.51	6.52
	Wanda	-	-	70.70	6.44
	D^2Prune	1	0	70.53	**6.36**
		0.5	0.5	**70.78**	6.38
60	SparseGPT	-	-	61.84	9.56
	Wanda	-	-	58.51	9.89
	D^2Prune	1	0	61.60	**9.05**
		0.5	0.5	**62.05**	9.11
70	SparseGPT	-	-	40.16	29.62
	Wanda	-	-	29.93	84.92
	D^2Prune	1	0	40.07	**21.10**
		0.5	0.5	**45.81**	21.48
80	SparseGPT	-	-	29.60	102.43
	Wanda	-	-	26.63	5107.20
	D^2Prune	1	0	28.43	**92.68**
		0.5	0.5	**29.81**	98.94

Table 9: Effect of λ_1, λ_2 settings under varying task distribution gaps (D^2Prune on LLaMA-2-7B).

Calibration samples. Figure 7 show the ablation study results on the calibration data sample of D^2Prune under 80% sparsity condition for all the LLMs, which demonstrates that the calibration data sample size of 128 is a reasonable setting.

B.5 Local Pruning vs. Global Pruning

To further evaluate the effectiveness of D^2Prune, we compare the performance of local and global pruning methods on the OPT-125M and LLaMA-2-7B models under varying sparsity levels. As shown in Table10, D^2Prune, a local pruning method that combines a Dual Taylor Expansion–based error estimator with attention distribution–aware dynamic weight update strategy, consistently outperforms global pruning baselines like SparseLLM(Bai et al. 2024) (SparseLLM is a recent global pruning framework that decomposes the global optimization objective into coordinated subproblems using auxiliary variables and closed-form solvers for MLP structure of Transformers). While it achieves strong performance, particularly in high-sparsity regimes, D^2Prune delivers superior results by minimizing local approximation errors more precisely and efficiently.

C Theoretical Analysis and Verification of Dual Taylor Expansion on Error

Assumption 1: $\delta\boldsymbol{x} = \lambda\boldsymbol{x}$ (in Eq. (8). As described in main text), due to the distributional difference between the calibration data and the original batch training data of pre-trained LLM, we cannot ignore the impact of activation variations on error. Specifically, consider a pre-trained large language model where the original input activation of a certain layer is $\boldsymbol{x}_0$, the weight is $\boldsymbol{w}_0$, and the output is $y = {\boldsymbol{x}_0}^T\boldsymbol{w}_0$. Now, suppose we remove a certain weight such that the error is $E = ||\boldsymbol{x}^T\boldsymbol{w} - {\boldsymbol{x}_0}^T\boldsymbol{w}_0||_2^2$, where $\boldsymbol{w} = \boldsymbol{w}_0 + \delta\boldsymbol{w}$, and $\boldsymbol{x} = \boldsymbol{x}_c = \boldsymbol{x}_0 + \delta\boldsymbol{x}$ is the input activation when using cali-

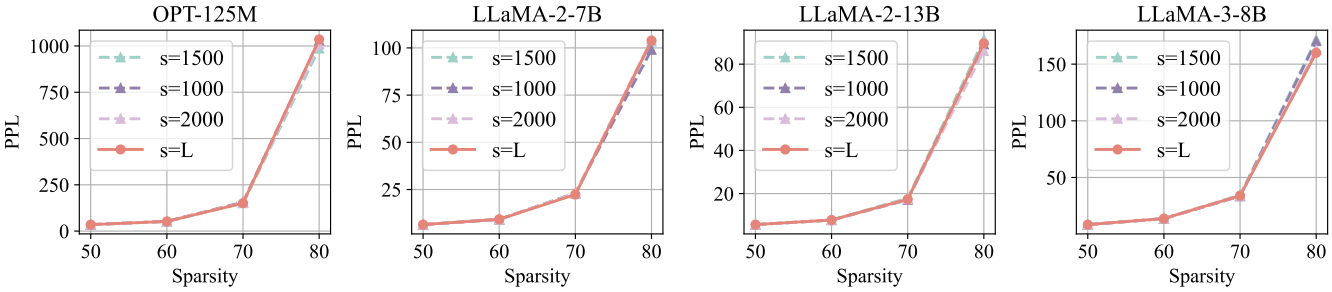


Figure 5: Perplexity change of different pruning models with varying scaling factor s on WikiText2

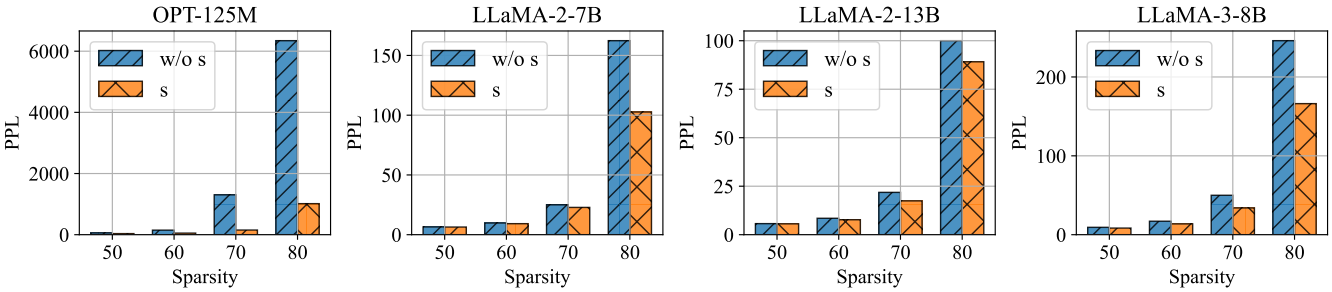


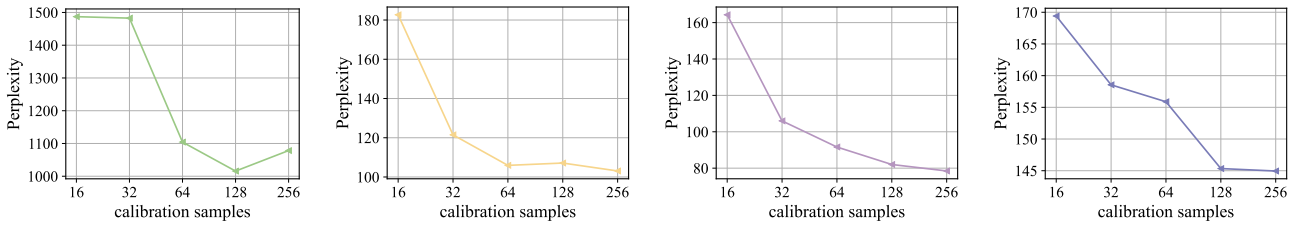
Figure 6: Comparison of model perplexity on WikiText2 before and after introducing the scaling factor s ("w/o s" denotes the setting without the scaling factor, while "s" indicates the setting with the scaling factor applied.

Model	Method	Sparsity			
		50	60	70	80
OPT-125M	SparseGPT	36.85	59.46	218.29	2140.55
	SparseLLM	36.11	56.64	172.85	1778.7
	D^2Prune	**34.98**	**52.1**	**160.81**	**1038.87**
LLaMA-2-7B	SparseGPT	7.02	10.23	26.78	108.11
	SparseLLM	7.06	10.26	28.87	118.22
	D^2Prune	**6.96**	**9.05**	**21.1**	**92.68**

Table 10: Perplexity of Global Pruning vs. Local Pruning for OPT-125M and LLaMA-2-7B on WikiText2. the lower the perplexity, the better. SparseGPT and D^2Prune are local pruning, while SparseLLM is global pruning. Our proposed pruning method D^2Prune outperforms SparseGPT and SparseLLM.

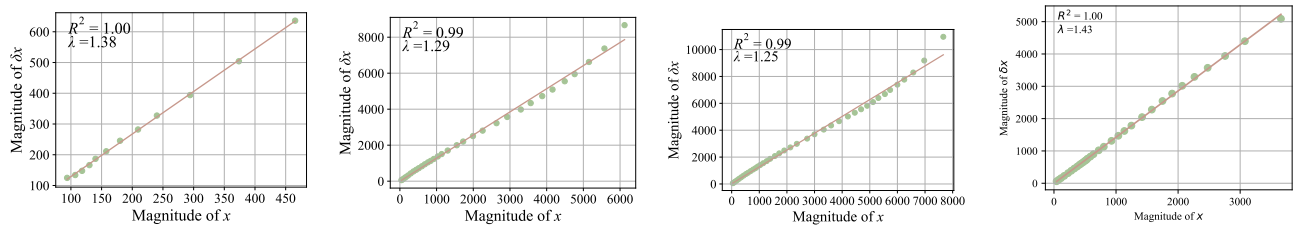
bration data. Therefore, $\delta\boldsymbol{x} = \boldsymbol{x} - \boldsymbol{x}_0 = \boldsymbol{x}_c - \boldsymbol{x}_0$. Relative to the original batch training data, when pruning with the calibration data, the activation change is determined by the difference in the distributions of both. However, the original training data is unknown but fixed, so the relationship of the difference of between is mainly determined by the calibration data. Ideally, $\boldsymbol{\delta x} = \boldsymbol{x} - \boldsymbol{x}_0 = \boldsymbol{x} + c$ (c is a constant) , exhibiting a linear relationship. Without loss of generality, assume that $\boldsymbol{\delta x} = \lambda \boldsymbol{x}$, λ is close to 1. To further verify the conjecture, we plot the layer-wise change of activation magnitude relationship under different data. As shown in Figure 8, 9, 10, we can see that when any two of the three different calibration datasets are used as model inputs, the layer-wise activation change δx exhibits a linear relationship with the activation x, with the fitted coefficient λ consistently close to 1. Theoretically, when $\lambda = 1$ (the default setting in our experiments), no additional hyperparameters are introduced in the partial derivative of the activation. To compensate for the perturbation caused by activation changes, we introduce the hyperparameters λ_1 and λ_2 in Eq. (10), which is a reasonable design choice. The experimental results in Table 8 also confirm the validity of the hyperparameter settings.

Assumption 2: The effect of the Hessian matrix cross terms is weak. In Eq. (7) (detailed in main text), we ignore the cross terms of the Hessian matrix, i.e., $\frac{1}{2}\delta\boldsymbol{w}^T H_{12}\delta\boldsymbol{x} + \frac{1}{2}\delta\boldsymbol{x}^T\boldsymbol{H}_{21}\delta\boldsymbol{w} = \delta\boldsymbol{w}^T H_{12}\delta\boldsymbol{x}$ (where $\boldsymbol{H}_{12} = \frac{\partial^2 E}{\partial\boldsymbol{w}\partial\boldsymbol{x}} = \boldsymbol{w}\boldsymbol{x}^T + \left(\boldsymbol{w}^T\boldsymbol{x} - y\right)\boldsymbol{I}$, $\boldsymbol{I}$ is the unit matrix). Since the pretrained model has $\frac{\partial E}{\partial \boldsymbol{w}} = \left(\boldsymbol{w}^T\boldsymbol{x} - y\right)\boldsymbol{x} = 0$ at local minima, we have $\boldsymbol{H}_{12} = \boldsymbol{w}\boldsymbol{x}^T$. Considering the presence of Layer Normalization in Transformers (i.e., $\mathrm{LayerNorm}\left(\boldsymbol{x}\right) = \frac{\boldsymbol{x}-\mu}{\sigma}$, μ, σ are the expectation and variance of the activation $\boldsymbol{x}$, respectively) , the input activations of each layer are normalized during the forward propagation on the calibration data, then $\mathbb{E}_{\boldsymbol{x}}\left[\boldsymbol{x}^T\right] = 0$. This leads to an expectation of 0 for the cross terms, i.e., $\mathbb{E}_{\boldsymbol{x}}\left[\boldsymbol{w}\boldsymbol{x}^T\right] = \boldsymbol{w}\mathbb{E}_{\boldsymbol{x}}\left[\boldsymbol{x}^T\right] = 0$. Therefore, the effect of the cross term $\boldsymbol{H}_{12}$ is weak and negligible compared to the Hessian matrix diagonal elements $\boldsymbol{H}_{11}$ and $\boldsymbol{H}_{22}$, which are higher-order infinitesimals. Even in the presence of unnormalized activations within the attention mechanism, we compensate for the errors introduced by non-zero cross terms of the Hessian matrix through dynamic adjustment of q, k, and v update configuration. The experimental results in Tables 2 and 3 (detailed in main text) further



(a) opt-125m (b) llama-2-7b (c) llama-2-13b (d) llama-3-8b

Figure 7: Perplexity sensitivity of different LLMs at 80% sparsity for the calibration samples.



(a) OPT-125M (b) LLaMA-2-7B (c) LLaMA-2-13B (d) LLaMA-3-8B

Figure 8: Layer-wise activation magnitude change relationship under different calibration data (C4-WikiText2). R^2 indicates the goodness of fit, the closer to 1, the better the fit is.

support this key insight. Compared to other methods such as SparseGPT and Wanda, our approach achieves better pruning performance without requiring additional Hessian-based computations (Just calculating the product term between the input and output activations paradigm with the weight magnitude, see 1), thereby validating the reasonableness of our assumption.

D Q, K, V Weights Update Configurations Search

In the process of optimizing neural network structures (see Figure 4), D^2Prune identifies the Q, K, and V update configurations that preserve the long-tail distribution characteristic of attention mechanisms, as shown in Table 11, Table 12 as well as Table 13.

Considering the cumulative effect of layer-wise reconstruction errors (also referenced in Eq. (12) (detailed in main text), we reduce the original Q/K/V configuration search space from $\sum_{k=0}^{3} C_3^k = 8$ combinations to 3 representative ones, and evaluate the corresponding perplexities. Under the constraint of model simplification, selecting the configuration that yields the lowest perplexity not only helps maintain inference performance after pruning but also strikes a favorable balance between model complexity and predictive capability, demonstrating the effectiveness of this pruning strategy.

Uniform Layer-wise Q, K, V Update Configuration Search. For unstructured pruning with layer-wise consistent Q, K, V update configurations search, only three full forward passes are needed to identify the optimal update strategy. Here, we report the optimal settings for each model as follows: OPT-125M, LLaMA-2-7B, LLaMA-2-13B, LLaMA-3-8B, and LLaMA-70B correspond to k: non-update, v: non-update, k: non-update, v: non-update, and k: non-update,

Method	Weight-Update	Layer	OPT-125M	LLaMA-2-7B	LLaMA-2-13B	LLaMA-3-8B
SparseGPT	✓	All	218.29	29.62	18.20	38.85
Wanda	✗	All	347.42	84.92	44.86	114.30
D^2Prune	✓	All	202.50	23.45	17.00	38.64
	✗	All	332.98	81.62	41.32	106.58
		q	198.46	25.16	17.32	37.14
		k	**153.97**	24.04	**16.51**	39.02
		v	203.60	**21.10**	17.19	**33.37**

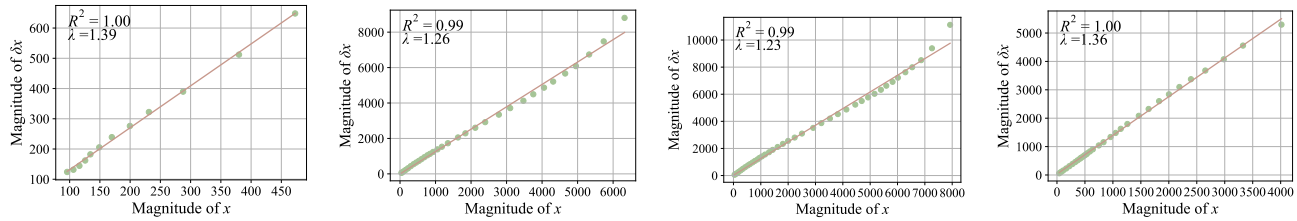
Table 11: Ablations study of dynamic weight update for attention modules (q, k, v) on WikiText2. (in perplexity, unstructured 70% sparsity)

Method	Weight-Update	Layer	OPT-125M	LLaMA-2-7B	LLaMA-2-13B	LLaMA-3-8B
SparseGPT	✓	All	59.46	9.56	7.77	14.40
Wanda	✗	All	74.39	9.89	7.87	22.80
D^2Prune	✓	All	55.39	9.46	7.58	13.66
	✗	All	71.55	9.75	7.66	22.54
		q	57.22	9.60	7.64	14.27
		k	**52.10**	9.55	**7.49**	14.36
		v	54.21	**9.05**	7.55	**13.44**

Table 12: Ablations study of dynamic weight update for attention modules (q, k, v) on WikiText2. (in perplexity, unstructured 60% sparsity)

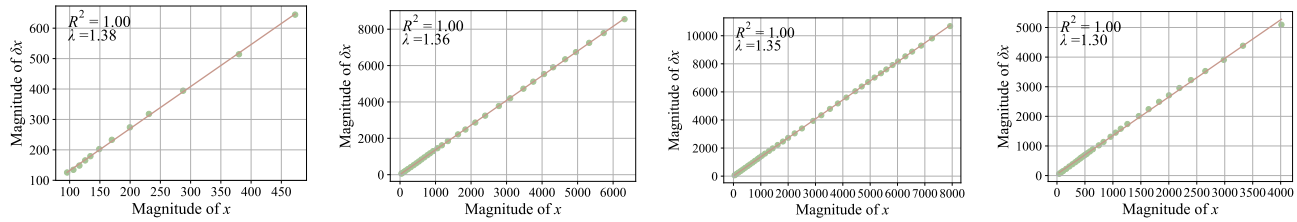
respectively (detailed experiments results in 3 (detailed in main text), Table 11, Table 12, Table 13). The configuration k: non-update indicates that, after pruning, only the q and v weights are updated, while the k weights remain fixed.

Non-Uniform Layer-wise Q, K, V Update Configuration Search. In the case of non-uniform Q, K, V update configuration search, each layer has three possible choices, leading to a total search space of 3^N for a model with N layers. This exponential complexity makes exhaustive search impractical. To mitigate this, we conduct a theoretical analysis of the relationship between Q/K/V update strategies and attention distribution error (detailed in Section "Attention Distribution-Aware Dynamic Weight Update Strategy"). A



(a) OPT-125M (b) LLaMA-2-7B (c) LLaMA-2-13B (d) LLaMA-3-8B

Figure 9: Layer-wise activation magnitude change relationship under different calibration data (C4-HellaSwag). R^2 indicates the goodness of fit, the closer to 1, the better the fit is.



(a) OPT-125M (b) LLaMA-2-7B (c) LLaMA-2-13B (d) LLaMA-3-8B

Figure 10: Layer-wise activation magnitude change relationship under different calibration data (WikiText2-HellaSwag). R^2 indicates the goodness of fit, the closer to 1, the better the fit is.

Method	Weight-Update	Layer	OPT-125M	LLaMA-2-7B	LLaMA-2-13B	LLaMA-3-8B
SparseGPT	✓	All	36.85	6.52	5.63	8.56
Wanda	✗	All	38.88	6.44	5.58	9.06
D^2Prune	✓	All	36.03	6.46	5.58	8.48
	✗	All	38.15	6.43	5.54	8.92
		q	36.01	6.50	5.60	8.55
		k	**34.98**	6.49	**5.53**	8.57
		v	36.25	**6.36**	5.56	**8.34**

Table 13: Ablations study of dynamic weight update for attention modules (q, k, v) on WikiText2. (in perplexity, unstructured 50% sparsity)

key insight is that preserving critical weights within Q, K, and V plays a vital role in maintaining the attention distribution. From this perspective, whether a Q/K/V matrix should be updated is highly related to its corresponding outlier distribution. In theory, the higher the weight outlier ratio, the more likely it contains essential weights. Updating such weights may severely disrupt the attention structure, and therefore, these components should be handled with greater caution. Let P denote the update probability of a given Q, K, or V weight matrix, and let r represent its corresponding outlier ratio. Based on our earlier analysis, in order to preserve critical weights, the update probability should be inversely related to the proportion of outliers. This relationship can be expressed as:

$$P \propto (1 - r)\,. \tag{17}$$

This relationship reflects an intuitive principle: when Q, K, or V weight matrix contains a higher proportion of outliers (i.e., larger r), its update probability should be correspondingly reduced to avoid disrupting the critical information encoded in the attention mechanism. To automatically determine the Q, K, V update configuration for each layer, inspired by OWL, we adopt a dual Taylor expansion-based error estimation (see Eq. (12) in main text) to compute the outlier score for Q/K/V weights. Specifically, we identify outliers as elements whose magnitudes is greater than the average value in each layer. For a certain weight $W \in \mathbb{R}^{m \times n}$ of Q/K/V, The outlier ratio r is defined as:

$$r = \frac{|\{w_{ij} \in W\} \, ||w_{ij}| > \mu|}{m \cdot n}, \tag{18}$$

where w_{ij} is the (i, j)-th element of the weight matrix W; $\mu = \frac{\sum_{i=1}^{m} \sum_{j=1}^{n} |w_{ij}|}{m \cdot n}$ is the mean of the absolute values of all elements in W; The numerator counts the number of elements whose magnitudes exceed μ , while the denominator is the total number of elements in the matrix W. To determine the update configuration for Q, K, and V in each layer, we first compute their respective outlier ratios by Eq. 18, denoted as r_Q, r_K, and r_V. Let $u(W) \in \{0, 1\}$ be an update indicator function for a weight matrix W, where $u(W) = 0$ indicates that W is not updated, and $u(W) = 1$ indicates that W is updated. We select the weight with the highest outlier ratio to remain unchanged, based on the intuition that preserving high-outlier weights helps maintain the attention distribution. Formally, the update rule is defined as:

$$u(W_i) = \begin{cases} 0, & \text{if } i = \underset{j \in \{Q,K,V\}}{\arg\max} \; r_j \\ 1, & \text{otherwise} \end{cases}, \tag{19}$$

where $W_i \in \{W_Q, W_K, W_V\}$. This strategy ensures that the most "outlier-dense" projection among Q, K, and V is preserved to minimize attention distortion.

We compute the layer-wise Q, K, and V update configuration based on Eq. 19, with experimental results on WikiText2 (in perplexity) summarized in Table 14. On one hand, both the uniform and outlier-based non-uniform Q, K, V update configuration achieve significantly lower perplexity than random assignment, validating the effectiveness and reliability

Sparsity	Layerwise Update	OPT-125M	LLaMA-2-7B	LLaMA-2-13B	LLaMA-3-8B
50	Uniform	34.98	6.36	5.53	8.34
	Random	35.50	6.42	5.55	8.44
	Non-Uniform	35.27	6.36	5.54	8.36
60	Uniform	52.10	9.05	7.49	13.44
	Random	54.13	9.28	7.42	41.08
	Non-Uniform	52.34	9.05	7.42	13.67
70	Uniform	160.81	21.10	16.51	33.37
	Random	174.45	23.00	16.62	35.21
	Non-Uniform	163.68	21.56	15.89	33.01
80	Uniform	1038.87	92.68	76.80	151.47
	Random	1382.89	107.19	80.43	187.50
	Non-Uniform	1081.53	96.18	76.12	171.16

Table 14: Comparison of uniform vs. non-uniform Layer-wise Q/K/V update vonfigurations across models at varying sparsity levels. "Random" denotes the random generation of Q/K/V updating configurations for each layer.

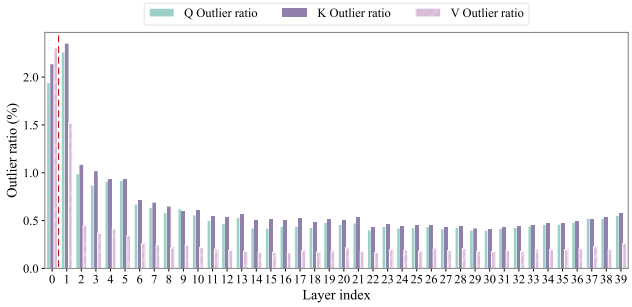


Figure 11: Layer-wise distribution of Q, K, and V outlier ratios in LLaMA-2-13B

of our method. On the other hand, the perplexity difference between the outlier-guided non-uniform and the uniform Q/K/V update configuration is surprisingly small.

At first glance, this observation appears counterintuitive. In theory, large language models exhibit heterogeneous sensitivity to Q, K, and V across layers, and their corresponding outlier distributions should vary accordingly. However, our results indicate that such variations do not translate into meaningful differences in pruning performance. To further analyze this phenomenon, we visualize the layer-wise outlier ratios of Q, K, and V in LLaMA-2-13B, as shown in Figure 11. Interestingly, despite visible distributional differences across layers, the K weight matrix consistently shows the highest outlier ratio in almost all layers, except for the first layer, where the V weight matrix slightly dominates. This suggests that the model inherently favors a near-uniform update configuration across Q, K, and V.

We believe this novel finding sheds light on the internal structure of attention in large language models and offers valuable insights into how Q, K, and V representations are organized and utilized.

E Pruing Efficiency and Speedup

Pruning efficiency. D^2Prune has a lower theoretical computational complexity than SparseGPT.We compare the empirical pruning speed of various methods by measuring the total pruning time on an NVIDIA A40 GPU. The results are shown in Table 15. Notably, our proposed method achieves better pruning performance than weight-update approaches like SparseGPT without introducing additional computational overhead. Compared to non-weight-update methods (Wanda and Pruner-Zero), our approach attains significantly higher accuracy under high sparsity with minor overhead.

Speedup. We evaluate the inference speedup enabled by semi-structured 2:4 sparsity on an NVIDIA A100 GPU. Following the evaluation protocols of SparseGPT (Frantar and Alistarh 2023) and Wanda (Sun et al. 2023), we measure the latency of matrix multiplications in the model's linear layers. The simulation is performed using high-performance GEMM kernels from the NVIDIA Cutlass library. Table 16 reports results on LLaMA-2-7B with batch size 1 and sequence length 2048. The results demonstrate that semi-structured 2:4 sparsity yields a significant inference speedup (approximately 1.3×) for the linear layers in large language models.

Method	OPT	LLaMA-2		LLaMA-3
	125M	7B	13B	8B
SparseGPT	49.13	963.68	1727.61	1893.83
Wanda	17.61	269.45	455.62	604.66
Pruner-Zero	17.50	268.93	504.84	617.72
D^2Prune	47.29	952.08	1617.51	1876.23

Table 15: Comparison of time overhead used for pruning models (in seconds).

LLaMA Layer	Dense	2:4	Speedup
q/k/v/o_proj	2.38	1.79	1.33×
up/gate_proj	3.29	2.43	1.35×
down_proj	1.66	1.25	1.32×

Table 16: Speedup of matrix multiplication (ms) in LLaMA-2-7B, for semi-structured 2:4 sparsity.

F Limitations and Future Works

Although D^2Prune optimizes the error minimization method in pruning, further improving the accuracy of pruning mask selection and weight updates, and making a significant step forward in the effective pruning for LLMs, attention-awareness for pruning mask selection and weight updates remains local suboptimal solutions. In our work, we still assume uniform sparsity, meaning that the pruning sparsity for each layer is the same and equal to the global sparsity. However, since different layers in LLMs have varying redundancy degree and pruning sensitivity, how to achieve an effective balance between sparsity and attention awareness, and use non-uniform sparse pruning for attention error compensation, is a significant direction. Finally, although we have explored post-training pruning compression techniques, we believe that considering instruction fine-tuning for different tasks and collaborative quantization compression will also be an important future research area.

G Related Work

Post-training pruning has emerged as a critical technique for compressing Large Language Models (LLMs) (Kwon et al. 2022; Zhang et al. 2024; Reyhan, Rahnamayan, and Bidgoli 2024), offering the advantage of eliminating the need for retraining while significantly reducing computational resource requirements. In this section, we briefly review post-training pruning techniques, categorizing them into the non-weight-update and weight-update pruning methods. Non-weight-update pruning defines a pruning metric to measure the importance of weights and then directly prunes unimportant weights without updating the remaining weights. For instance, Wanda (Sun et al. 2023) introduces an improved magnitude-based criterion that combines activations with weights. Building upon this, Pruner-Zero (Dong et al. 2024) employed symbolic regression and genetic programming to automate the discovery of pruning evaluation functions. On the other hand, weight-update pruning methods are generally based on (OBS) (Hassibi, Stork, and Wolff 1993) for pruning mask selection and weight update, aiming to minimize layer-wise errors introduced by pruning. Typically, SparseGPT (Frantar and Alistarh 2023) utilizes Hessian matrices and calibration data to select masks and update weights.

Recent advances have further improved weight-update pruning. The ADMM-Grad method (Boža 2024) introduces an alternating direction method of multipliers (ADMM) for fast and effective weight updates after pruning, achieving state-of-the-art results across various LLMs. Meanwhile, LLM Surgeon (van der Ouderaa et al. 2023) offers a general framework for unstructured, semi-structured, and structured pruning. It leverages Kronecker-factored curvature approximations to dynamically allocate pruning across layers and update weights efficiently, delivering top-tier performance in structured pruning of LLMs. SparseLLM (Bai et al. 2024) optimizes the pruning process of MLP modules in LLMs decoder layer through a global optimization approach.